

基于粒子群算法（PSO）的烟幕干扰弹的投放策略模型

摘 要

在现代军事防御作战中，当我方目标被敌方导弹锁定时，无人机投放烟幕弹遮蔽该目标作为低成本且高效的防御手段至关重要。本论文将针对在不同环境下无人机携带烟幕弹对抗来袭导弹的问题，构建系列数学模型并求解优化策略。

针对问题一，在给定无人机和导弹初始条件下，通过建立三维运动学模型，确定单枚干扰弹对导弹的有效遮蔽时长，属于动态几何交汇问题。采用数值遍历方法计算云团有效时窗内的遮蔽区间，最终得出有效遮蔽时长为 1.4s。针对问题二，以最大化单枚烟幕弹对导弹的遮蔽时长为目标，属于多变量约束优化问题。采用粒子群优化（PSO）算法，通过动态惯性权重平衡全局与局部搜索在解空间搜索最优参数组合，该算法能有效处理多变量优化，最终得到最大遮蔽时长 4.72s。针对问题三，在单机多弹模式下，建立多弹协同优化模型，属于多阶段决策优化问题。采用改进 PSO 算法（引入随机扰动破除局部最优），求解得总有效遮蔽时长 6.75s。针对问题四，在三机单弹协同场景下，构建协同优化模型，以三枚干扰弹对单导弹的总遮蔽时长最大化为目标，属于多目标协同优化问题。采用分治 PSO 算法，降低算法复杂度，为每架无人机独立优化参数并协同组合，最大化总遮蔽时长，实现遮蔽区间有效互补，最终求得总有效遮蔽时长 14.52s。针对问题五，在五机多弹对抗三枚导弹场景下，以三枚导弹同时被遮蔽的总时长最大化为目标，构建混合整数规划模型（上层 0-1 规划分配干扰目标，下层 PSO 优化参数），也属于多目标协同优化问题。通过分阶段求解，第一阶段使用分群 PSO，第二阶段使用贪心组合筛选和 0-1 规划，第三阶段使用并行化局部爬山算法最终实现多机多弹对三导弹的协同遮蔽，有效提升整体防御效能，最终求得总有效遮蔽时长 25s。

所建模型可推广至其他电子对抗领域，如无人机集群路径规划、多目标火力分配等。模型优点在于采用三维建模准确描述实体动态交互，PSO 算法无需梯度信息可处理高维问题，且模型具有可传递性。缺点是未充分考虑环境等因素影响，PSO 算法容易出现早熟，高维情况下难以保证找到全局最优解。未来可考虑更多实际因素，改进算法，如融合模拟退火和遗传算法以提高求解精度。

关键词 粒子群优化算法（PSO）；多目标优化；整数规划；动态规划；

一、问题重述

1.1 问题背景

在现代防御作战中，来袭导弹往往以高速直指目标，常规拦截代价高且成功率有限。烟幕干扰作为一种成本较低的末端防御手段，能够在导弹飞行末段形成局部遮蔽区域，从而干扰导弹的末制导识别与跟踪。为了提升干扰效果，通常采用无人机携带干扰弹，在适当位置和时机释放，使得烟幕云团与导弹飞行路径重叠并维持一定时长。

在实际场景中，干扰弹通常具有延时起爆特性。投放后受重力作用运动，起爆时瞬间形成一定半径的烟幕云团，并在有限时间内有效。无人机飞行速度和方向、干扰弹的投放时机与延迟设定，都会直接影响烟幕与导弹路径的交叠情况，进而决定遮蔽时间长短。因此，如何设计无人机的飞行轨迹和干扰弹的投放方案，使得导弹被有效遮蔽的时长尽可能长，是一个亟需解决的问题。

1.2 问题回顾

问题 1：在给定无人机和导弹初始位置、速度及投放与起爆条件下，建立数学模型，计算单枚干扰弹能否对导弹形成有效遮蔽，并给出遮蔽持续时间。

问题 2：在无人机仅能投放一枚干扰弹的情况下，建立数学模型，分析飞行速度、飞行方向、投放时刻与起爆延迟等因素对遮蔽时间的影响，并确定最优投放方案。

问题 3：当无人机最多可投放三枚干扰弹时，建立数学模型，安排合理的投放时刻与起爆延迟，使导弹的累计遮蔽时间尽可能长。

问题 4：当三架无人机分别投放一枚干扰弹时，建立数学模型，研究多机协同作用，并设计飞行与投放方案，使导弹的遮蔽时间最大化。

问题 5：当有五架无人机（每架最多三枚干扰弹）对抗三枚同时来袭的导弹时，建立数学模型，进行投放任务分配与调度，使三枚导弹被有效遮蔽的总时长最大。

二、问题分析

2.1 问题一分析

问题一是一个动态规划和最优控制问题，需要实现无人机投放烟幕干扰弹以干扰来袭导弹的数学建模与策略优化。本问仅需考虑单机单弹对单弹情况下的干扰问题，要求在 FY1 在确定飞行速度、航向、投放时间和起爆延时条件下，求解该枚烟幕弹对 M1 的有效遮蔽时长。首先，我们我们在三维坐标系中构建导弹匀速直线运动、烟幕弹平抛运动及云团匀速下沉的运动学方程，接着通过计算导弹轨迹与动态云团之间的最小距离，确定导弹进入和离开有效遮蔽区域的时间点。最后采用数值方法遍历云团有效的 20 秒时长，从而求解出遮蔽时间区间和有效遮蔽时长^[1]。

2.2 问题二分析

问题二要求在无人机 FY1 单枚烟幕弹干扰 M1 的基础上，通过优化无人机

飞行策略和弹道参数实现遮蔽时间最大化。这是一个典型的多变量约束优化问题，需要协调无人机飞行方向、飞行速度、投放时机和起爆时机四个决策变量的组合，使其产生的动态烟幕云团能与导弹轨迹形成最长时间的交汇。首先要定义决策变量及其参数范围，之后要基于定义的决策变量，重新构建导弹、无人机、烟幕干扰弹及云团的运动方程，然后要建立以有效遮蔽时长为目标函数，以运动学方程和战术约束为条件的优化模型，采用智能优化算法在解空间中搜索最优参数组合，通过迭代计算不同参数组合下的遮蔽时长，根据适应度值不断更新搜索方向，最终收敛到全局最优解。最后，对结果进行结果分析。

2.3 问题三分析

问题三在问题二的基础上增加了烟幕干扰弹数量，由单机单弹对单弹到单机三单对单弹，要求通过优化多弹投放时序和起爆策略实现遮蔽时间最大化。首先，提出“分时段协同干扰策略”，将三枚烟幕弹的投放时间按先后顺序分区段规划，形成时序上的连续覆盖。然后，以总有效遮蔽时长最大化为目标，建立多弹协同优化模型，采用改进粒子群算法求解最优的无人机飞行参数和三枚弹的投放起爆参数。最后，通过数值仿真验证多弹协同策略的有效性，并对结果进行对比分析。

2.4 问题四分析

问题四在问题二的基础上转化为多机协同干扰，无人机空间分布和各个参数各不相同，这意味着每架无人机都需要采用独特的飞行策略，在建模过程中需要为各无人机独立决策其飞行方向、飞行速度和弹道参数，最终目标是使三枚烟幕弹形成的遮蔽时间段在时间轴上形成有效互补，通过优化算法求解使三架无人机发射的干扰弹的遮蔽时间并集达到最大化，从而实现多机协同干扰的最优效果。

2.5 问题五分析

问题五在问题四多机协同干扰单目标的基础上，进一步扩展为多机多弹干扰多目标的全域协同作战场景，需要 5 架无人机实现对 3 枚导弹的同时有效干扰。与问题四仅需最大化单弹遮蔽时长不同，本题要求必须确保三枚导弹在同一时刻均被遮蔽，任何一枚导弹的失防都将导致该时刻干扰失效。本文通过严格定义“三弹同时遮蔽”为时间交集，建立包含离散弹目分配和连续弹道参数优化的混合整数规划模型，将问题分解为上层弹目分配与下层弹道参数优化两个层次，上层通过 0-1 规划确定每枚烟幕弹的最优干扰目标，下层采用改进粒子群算法优化各无人机的飞行参数和弹道参数，在确保每枚导弹至少获得一枚干扰弹覆盖的前提下，最大化三弹同时被遮蔽的有效时长。

三、模型假设

为了方便模型的建立与模型的可行性，我们这里首先对模型提出一些假设，使得模型更加完备，预测的结果更加合理。

- 1、任何未在题目中明确记录的外部因素（如风速、天气等）对预测结果的影响被忽略。
- 2、数据完整性假设：假设题目所给的导弹和无人机的初始位置、速度等数据是准确且完整的，不存在缺失值或记录错误。
- 3、导弹飞行假设：来袭导弹以 300 m/s 匀速直线飞行，始终指向假目标，不发

生规避或机动。

4、无人机飞行假设：无人机在受领任务后可瞬时调整航向，随后以 $70 - 140 \text{ m/s}$ 的等高度匀速直线飞行；一旦航向和速度确定，就不再改变。

5、烟幕干扰弹假设：干扰弹投放瞬间具有无人机相同的水平速度，之后仅受重力作用，不考虑空气阻力和风场影响。

6、烟幕云团假设：干扰弹起爆后瞬间形成球形云团，半径固定为 10 m ，并以 3 m/s 的速度匀速下沉，在起爆后 20 s 内保持有效遮蔽能力。此时，水平速度为 0 ，不考虑风速、空气阻力等外部影响。

7、遮蔽判定假设：若导弹在某一时刻进入任意一个云团球心半径 10 m 内，且未超过云团 20 s 的有效时长，则视为导弹被有效遮蔽，不叠加计算重复遮蔽。

四、符号说明

为了方便我们模型的建立与求解过程，我们这里对使用到的关键符号进行以下说明：

符号	符号说明	单位
D_i	导弹 M_i 初始位置	m
v_M	导弹 M_i 飞行速度	m/s
D_{FYk}	无人机 FY_k 初始位置	m
g	重力加速度	m/s^2
v_{FYk}	无人机 FY_k 飞行速度	m/s
t_{rel}	干扰弹投放时刻	s
Δt	干扰弹起爆延迟	s
t_{exp}	干扰弹起爆时刻	s
v_s	云团下沉速度	m/s
r	云团有效半径	m
Q	真目标代表点	m
u_M	导弹指向原点的单位向量	—
$M_i(t)$	导弹在 t 时刻的位置	m
u_F	无人机指向原点的单位向量	—
$F(t)$	无人机在 t 时刻的位置	m
C	起爆点代表点	m
θ	无人机方向角	rad

五、模型的建立与求解

5.1 通用运动学模型的建立

在无人机防御场景中，我们考虑三个主要实体：无人机（作为目标或发射平台）、导弹（作为攻击者）和干扰弹（作为防御手段）。为了建立轨迹方程，我们基于牛顿运动定律和几何光学原理，提出以下通用模型。

5.1.1 无人机轨迹方程

我们首先定义通用无人机匀速直线运动模型。对于无人机（编号 $k=1,2,\dots,5$ ），其位置随时间变化可表示为：

$$F_k(t) = D_{FYk} + v_k \cdot t \cdot (\cos\theta_k, \sin\theta_k, 0) \quad (1)$$

其中， D_{FYk} 是初始位置向量， v_k 是常速度向量， θ_k 是飞机的方向角。

5.1.2 导弹轨迹模型

导弹（编号 $i=1,2,3$ ）被建模为以恒定速度朝向假目标 $O(0, 0, 0)$ 运动。

$$M_i(t) = D_{ij} + v_M \cdot u_{Mi} \cdot t \quad (2)$$

其中， u_{Mi} 是每个导弹指向原点的单位向量。

$$u_{Mi} = -\frac{D_{ij}}{\|D_{ij}\|} \quad (3)$$

5.1.3 干扰弹及云团运动模型

干扰弹由无人机投放，其运动分为两个阶段：平抛阶段和云团下沉阶段。每个干扰弹（索引 j ）独立运动，互不干扰。

在投放时刻 $t_{rel,kj}$ 到起爆时刻 $t_{exp,kj}$ 之间为平抛运动，投放点可表示为：

$$P_{drop,kj} = F_k(t_{rel,kj}) = \begin{cases} D_{FYk,x} + v_{FYk} \cdot t_{rel,kj} \cdot \cos\theta_k \\ D_{FYk,y} + v_{FYk} \cdot t_{rel,kj} \cdot \sin\theta_k \\ D_{FYk,z} \end{cases} \quad (4)$$

由于惯性，烟幕干扰弹在投放时刻之后，水平方向上仍保持无人机的速度 v_{FYk} 做匀速运动，在竖直方向上做自由落体运动，则起爆点坐标 $P_{exp,kj}$ 可以表示为：

$$P_{exp,kj} = \begin{cases} P_{drop,kj,x} + v_{FYk} \cdot \Delta t_{kj} \cdot \cos\theta_k \\ P_{drop,kj,y} + v_{FYk} \cdot \Delta t_{kj} \cdot \sin\theta_k \\ D_{FYk,z} - \frac{1}{2} g \Delta t_{kj}^2 \end{cases} \quad (5)$$

其中， $t_{expkj} = t_{relkj} + \Delta t$ ， Δt 是平抛时间， g 是重力加速度。

起爆之后，烟幕干扰弹瞬时形成球状烟幕云团，由于巨大的阻力，云团失去水平方向的速度，竖直方向以 $3m/s$ 匀速下沉，则云团中心的轨迹可以表示为：

$$C_{kj}(t) = \begin{cases} P_{exp,kj,x} \\ P_{exp,kj,y} \\ P_{exp,kj,z} - v_{sink} \cdot (t - t_{expkj}) \end{cases} \quad (6)$$

5.1.4 视线遮蔽条件模型

为了判断导弹视线是否被云团遮蔽，我们参数化“导弹 $M(t)$ -真目标 Q

” 视线。视线上的任意点可表示为：

$$L_{kj}(s) = Q + s \cdot (Mi(t) - Q) \quad (s \in [0,1]) \quad (7)$$

视线在云团内时可有效遮蔽，即视线到云团中心最短距离小于 10m，

$$\|L_{kj}(s) - C_{kj}(t)\| \leq 10 \quad (8)$$

综上所述，本文所涉及的无人机匀速直线运动、导弹定向攻击、干扰弹平抛与云团下沉等物理过程，均可由上述通用运动学模型完整描述。为更直观地展示各实体间的运动关系与关键的遮蔽判定几何，本文构建了各实体运动轨迹与遮蔽关系示意图，如图 1 所示：

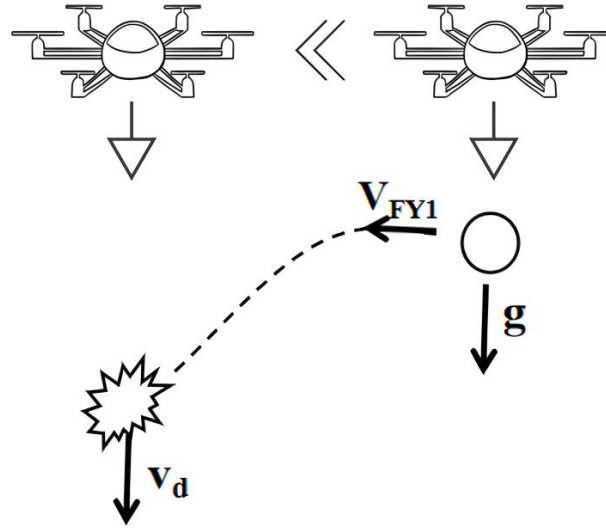


图 1：防御系统各实体运动轨迹与遮蔽关系示意图

该通用模型是后续所有问题分析与求解的物理基础。基于此，我们将针对各个问题的具体约束与优化目标，进行模型的专项化建立与求解。

5.2 问题一的模型建立与求解

5.2.1 问题一的求解思路

问题一是典型的动态规划问题，首先在三维坐标系内建立导弹、无人机和烟幕弹的三维运动学模型，其次计算烟幕弹的起爆点位置，然后分析云团形成后的动态运动特性；最后通过数值方法计算导弹视线与云团有效区域的空间交汇时间窗口，得出遮蔽时间区间和有效遮蔽时长。

5.2.1 问题一模型的建立

首先建立以假目标为原点的三维坐标系，已知烟幕干扰弹脱离无人机后做自由落体运动，起爆后形成的球状云团以 3m/s 匀速下沉，有效遮蔽范围为云团中心 10m 范围内，持续时间 20 秒。导弹以 300m/s 速度直线飞向假目标。无人机可在受领任务后瞬时调整航向，以 70~140m/s 的速度等高度匀速飞行。每架无人机每次投放两枚烟幕弹需间隔至少 1 秒。根据以上已知条件，可以得出导弹 M1 的轨道方程、无人机 FY1 轨迹方程^[2]。

(1) 导弹 M1 和无人机 FY1 的轨道方程

根据导弹 M1 初始位置(20000,0,2000)和飞行速度 V300m/s，代入公式 (2) 推导出导弹 M1 的轨迹方程为：

$$M_1(t) = D_1 + v_M \cdot u_{Mi} \cdot t = (20000,0,2000) + 300t \cdot u_{M1} \quad (9)$$

同理，根据无人机的初始位置 FY1(17800, 0, 1800)代入公式(1)得出无人机 FY1 在 t 时刻的位置方程： $F_1(t) = (17800, 0, 1800) + 120t \cdot (\cos \theta_1, 0, 0)$ 。

(2) 干扰弹投放与起爆点

若 FY1 以 120 m/s 的速度朝向假目标方向飞行，受领任务 1.5 s 后即投放 1 枚烟幕干扰弹，把 $t_{rel} = 1.5s$ 代入公式（4）得投放无人机位置 $P_{drop,11} = F_1(1.5)$

干扰弹在间隔 3.6 s 后起爆，这期间做水平初速度为 120m/s 的平抛运动，代入公式（5）得出起爆点的坐标 $P_{exp,11}$ 。

(3) 云团中心轨迹

起爆后云团中心以 $v_s = 3m/s$ 匀速下沉， $t \geq 5.1s$ 时，云团中心轨迹可用 t 来表示

$$C_{11}(t) = (P_{exp,11,x}, P_{exp,11,y}, P_{exp,11,z} - 3(t - t_{exp11})) \quad (10)$$

(4) 视线线段与云团相交判定

根据公式(7)得出

$$L_{11}(s) = Q + s \cdot (M1(t) - Q) \quad (s \in [0,1]) \quad (11)$$

$$\|L_{11}(s) - C_{11}(t)\| \leq 10 \quad (12)$$

5.2.3 问题一求解与结果分析

通过编程实现给出每个 t 的最小距离，满足距离小于等于 10 的 t 构成若干时间区间，其收集长度即为“有效遮蔽时长”。

为直观展示问题一的干扰效果，图 2 绘制了导弹 M1 运动轨迹、无人机 FY1 飞行轨迹及烟幕云团运动轨迹的三维空间关系。如图 2 所示，烟幕弹在指定参数下完成投放与起爆后，形成的下沉云团成功拦截了导弹指向真目标的视线，并得出了有效遮蔽时区和有效遮蔽时长。

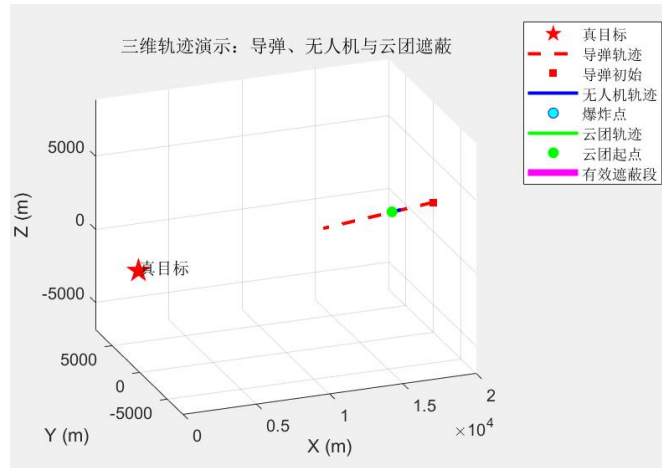


图 2：问题一三维轨迹演示图

通过计算，得到：有效遮蔽时间区间(s):[8.0400,9.4400]；有效遮蔽时长 = 1.4 s。

5.3 问题二的模型建立与求解

5.3.1 问题二的求解思路

第二问在第一问的基础上，添加了无人机飞行方向、无人机飞行速度、烟幕弹投放时刻、烟幕弹起爆延时等决策变量，首先要确定决策变量和约束条件，然后三维建模重新推导导弹、无人机、烟幕干扰弹和烟幕云团的三维坐标方程，之后采用粒子群优化（PSO）算法进行求解，通过群体智能搜索机制在解空间中寻

找全局最优参数组合，最终输出最大化遮蔽时长的最优干扰策略。

5.3.2 问题二的模型建立过程

在问题一的基础上，添加了无人机 FY1 方向角 θ 、无人机飞行速度 v_{FY1} （70-140 m/s 范围内）、烟幕弹投放时刻 $t_{rel,k,1}$ 、烟幕弹起爆延时 Δt 四个决策变量，并添加了无人机速度约束，时间非负约束和集合约束条件，要求无人机速度在 70m/s 到 140m/s 之间，保证 $t_{rel} \geq 0$ ， $\Delta t > 0$ ，同时保证云团需在“导弹-真目标”实现范围内形成遮挡。

粒子群算法通过设计一种无质量的粒子来模拟鸟群中的鸟，粒子仅具有两个属性：速度和位置，速度代表移动的快慢，位置代表粒子当前所在的解，非常适合求维连续决策变量优化问题^[4]。针对这个多决策优化问题，我们选择 PSO 粒子群优化算法进行求解，其算法流程如图 3 所示：

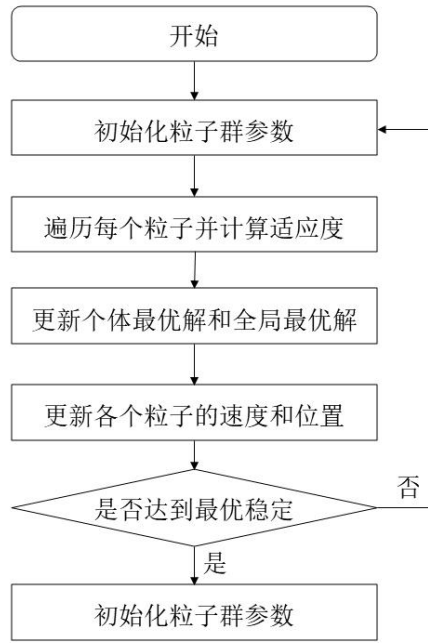


图 3: POS 粒子群算法流程图

我们采用 PSO 粒子群算法建模，用每个粒子 X 表示一组决策变量， θ 表示无人机飞行方向角，相对于 +x 轴逆时针方向。

$$X = [\theta, v_{FY1}, t_{rel}, \Delta t] \quad (13)$$

$$(\theta \in [-\pi, \pi], v_{FY1} \in [70, 140], t_{rel} \geq 0, \Delta t \in [0.1, 15])$$

基于新决策变量，重新推导导弹、无人机、干扰弹及烟幕云团的时空位置方程，其核心机理符合 5.1 中的通用模型。

有效遮蔽时长 T_{shield} 为有效时间窗内遮蔽有效的总时间，可以用积分表示为：

$$T_{shield} = \int_{t_{exp}}^{t_{exp}+20} I(\text{视线被云团遮蔽}) dt \quad (14)$$

其中， $I(\cdot)$ 为指示函数，遮蔽时间有效时取 1，否则取 0。

由以上公式，可得出需要优化的目标函数为：

$$\max T_{shield}(\theta, v_{FY1}, t_{rel}, \Delta t) \quad (15)$$

首先随机生成 N 个粒子，初始化速度与位置，设置个体最优 p_{best} 和全局最优 g_{best} 。对每个粒子，计算其有效遮蔽时长，更新优化粒子的 p_{best} 和群体的 g_{best} 。速度与位置迭代更新过程可表示为：

$$\begin{cases} v_i^{k+1} = w \cdot v_i^k + c_1 r_1 (p_{best,i} - x_i^k) + c_2 r_2 (g_{best,i} - x_i^k) \\ x_i^{k+1} = x_i^k + v_i^{k+1} \end{cases} \quad (16)$$

其中, w 为惯性权重, c_1, c_2 为学习因子, r_1, r_2 为随机数, p_{best} 为个体最优解, g_{best} 为全局最优解。

同时, 为保证解的可行性, 需要对更新后的粒子位置进行边界约束修复, 当生成的粒子超过边界的时候, 将粒子截断到边界处:

$$x_{i,j}^{k+1,clamped} = \begin{cases} lower_j & x_{i,j}^{k+1} < lower_j \\ x_{i,j}^{k+1} & lower_j \leq x_{i,j}^{k+1} \leq upper_j \\ upper_j & x_{i,j}^{k+1} > upper_j \end{cases} \quad (17)$$

最终对粒子 X 进行适应度计算, 将有效时长 T_{shield} 作为适应度, 即是目标函数值。

5.3.3 问题二的结果

根据所建立的优化模型及 POS 算法求解, 我们可以得到基于 PSO 算法的决策变量呈收敛趋势如图 4 所示, 表明优化过程有效。

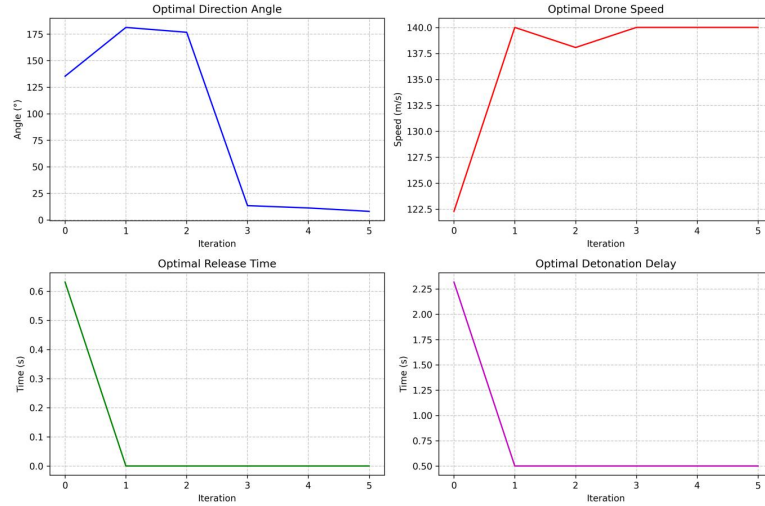


图 4: 基于 PSO 算法优化过程的决策变量迭代收敛曲线

基于 PSO 优化过程得到的 T_{shield} 适应度曲线也在多次迭代之后稳定收敛, 示意图如图 5 所示:

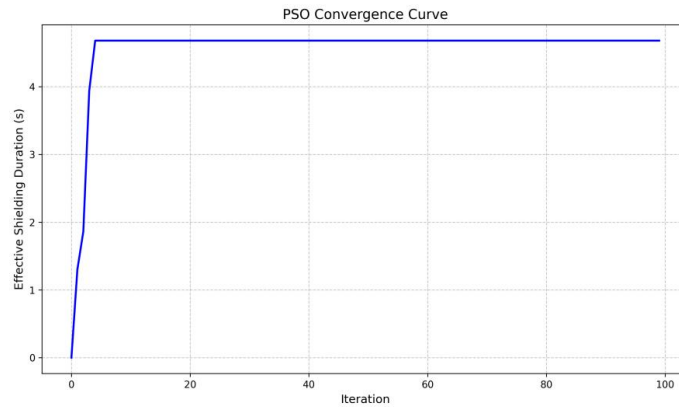


图 5: 基于 PSO 算法优化过程的适应度函数 T_{shield} 迭代收敛曲线

将单无人机三弹对单弹的最优作战策略使用三维状态仿真模拟如图 6 所示:

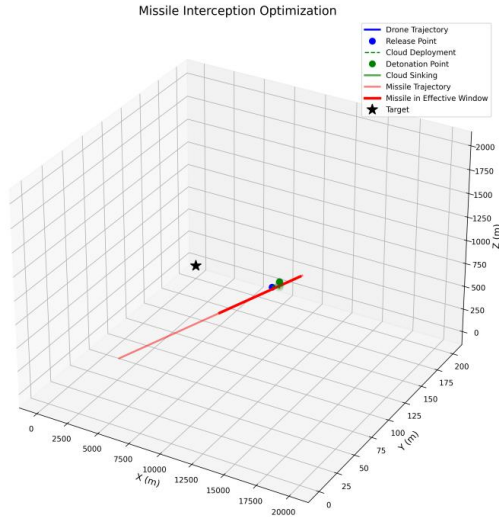


图 6：无人机三弹协同干扰三维轨迹仿真图

经优化算法求解，我们得到无人机 FY1 单枚烟幕弹干扰导弹 M1 的最优策略参数如下：

最优方向角($^{\circ}$): 5.10167668029398
 最优无人机速度(m/s): 93.30166983427607
 最优投放时刻(s): 0.6507860901001019
 最优起爆延迟(s): 0.5
 最大有效遮蔽时长(s): 4.719999999999987
 投放点坐标 (X, Y, Z): (17860.478886684527, 5.3993807709522805, 1800.0) m
 起爆点坐标 (X, Y, Z): (17906.944912621886, 9.547733703727598, 1798.8) m

5.4 问题三模型求解与建立

5.4.1 问题三的求解思路

问题三要求在单机多弹模式下实现对导弹 M1 的最优干扰，是一个多阶段决策优化问题。与问题二相比，复杂度显著提升，需要协调多枚烟幕弹的投放时序、空间分布和起爆策略，形成协同干扰效应。首先要建立多弹协同优化模型^[5]，此处无人机仅有一架，省略无人机编号，定义无人机速度 v 、航向角 θ 、三枚弹的投放时间 $t_{rel,1}$ 、 $t_{rel,2}$ 、 $t_{rel,3}$ （满足 $\Delta T \geq 1s$ ）、起爆延时 $\Delta t_1, \Delta t_2, \Delta t_3$ ，然后建立整数规划约束条件，采用改进的多目标粒子群优化算法，通过引入动态惯性权重和自适应学习因子，提高算法在 8 维解空间中的搜索效率。

5.4.2 问题三模型建立过程

5.4.2.1 数学模型的建立

问题三是在问题二的基础上增加了决策变量，导弹轨迹、无人机轨迹方程与问题二一致，需要分阶段建立多枚烟幕弹的运动模型。

三枚烟幕干扰弹在投放前共享无人机运动轨道，此时对于无人机在 $t_{rel,k}$ 时刻投放第 k 枚烟幕干扰弹的位置 $P_{drop,k}$ 可表示为：

$$P_{drop,i} = F(t_{rel,i}) = \begin{cases} 17800 + v_{FY1} \cdot t_{rel,i} \cdot \cos\theta \\ 0 + v_{FY1} \cdot t_{rel,i} \cdot \sin\theta \\ 1800 \end{cases} \quad (18)$$

由于惯性，烟幕干扰弹在投放时刻之后，水平方向上仍保持无人机的速度 v_{FY1} 做匀速运动，在竖直方向上做自由落体运动，当到达 $t_{exp,k}$ 时刻时，干扰弹起爆，此时三枚干扰弹的坐标 $P_{exp,k}$ 可以表示为：

$$P_{exp,i} = \begin{cases} P_{drop,i,x} + v_{FY1} \cdot \Delta t_i \cdot \cos\theta \\ P_{drop,i,y} + v_{FY1} \cdot \Delta t_i \cdot \sin\theta \\ 1800 - \frac{1}{2}g\Delta t_i^2 \end{cases} \quad (19)$$

其中， $P_{drop,i}$ 表示第 i 枚烟幕干扰弹的投放点， v_{FY1} 表示无人机飞行速度， θ 表示无人机飞行方向角， Δt_i 表示第 i 枚烟幕干扰弹投放到起爆的时间， $t_{exp,ik} = \Delta t_i + t_{rel,i}$ 。

起爆之后，烟幕干扰弹瞬时形成球状烟幕云团，由于巨大的阻力，云团失去水平方向的速度，竖直方向以 $3m/s$ 匀速下沉，则云团中心的轨迹可以表示为：

$$C_i(t) = \begin{cases} P_{exp,i,x} \\ P_{exp,i,y} \\ P_{exp,i,z} - v_{sink} \cdot (t - t_{exp,i}) \end{cases} \quad (20)$$

题目要求任意两枚干扰弹投放间隔要大于等于 $1s$ ，所以约束条件应增加投放时间间隔约束：

$$t_{rel,2} - t_{rel,1} \geq 1, t_{rel,3} - t_{rel,2} \geq 1 \quad (21)$$

与单弹情况不同，多弹遮蔽存在时间重叠和空间互补的特性，因此不能简单地将各弹的遮蔽时间累加，应采用时间并集的概念来定义总有效遮蔽时长，即只要在某一时刻至少有一枚烟幕弹形成有效遮蔽，该时刻就被计入总遮蔽时间。

可设全局时间轴范围为 $[t_{min}, t_{max}]$ ，($t_{min} = \min(t_{exp,1}, t_{exp,2}, t_{exp,3})$, $t_{max} = \max(t_{exp,1} + 20, t_{exp,2} + 20, t_{exp,3} + 20)$)，则目标函数可表示为：

$$\max T_{total} = \int_{t_{max}}^{t_{min}} I \left(\bigcup_{i=1}^3 (t \in [t_{exp,i}, t_{exp,i} + 20] \cap d_i(t) \leq 10) \right) dt \quad (22)$$

其中， $I(\cdot)$ 为指示函数，遮蔽时间有效时取 1，否则取 0； $\cup$ 表示时间并集， $d_k(t)$ 为 t 时刻弹 k 的视线-云团最短距离。

5.4.2.2 数学模型的求解

我们采用改进型粒子群优化 PSO 算法求解目标函数，烟幕弹增加到三个之后，决策变量个数从 4 个变成 8 个，每个粒子对应决策变量的一组解：

$$X = [\theta, v_{FY1}, \Delta t_1, \Delta t_2, \Delta t_3, t_{rel,1}, t_{rel,2}, t_{rel,3}] \quad (23)$$

($\theta \in [0, 2\pi]$, $v_{FY1} \in [70, 140]$, $t_{rel,k} \in [0, 50]$, $\Delta t_1 \in [10^{-6}, 15]$)

在解决问题二时，全局最优适应度出现“早熟收敛”，即过早地找到了一组还不错的参数，从而无法继续搜索遮蔽时长更大的更好参数，陷入局部最优困扰，因此我们引入了双重改进机制：

(1) 采用非线性变化的惯性权重结合随机扰动项

$$w = \left(0.8 - 0.6 \cdot \frac{iter}{\max_iter} \right) + 0.05 \cdot randn() \quad (24)$$

将原本匀速变化的惯性权重 w 上面添加一个随机抖动 $randn()$ ，从而破除局部最优困扰。

(2) 添加全局最优变异

当算法检测到连续 20 次迭代全局最优 g_{best} 适应度变化不超过 $1e-4$ ，即长时间内“最好成绩”都没有显著提升时，就判断发生“早熟收敛”，此时我们强

行改变当前全局最优解 g_{best} 中的参数，对低敏感度变量（如 $\Delta t_1, \Delta t_2$ ）变异：

$$g_{best}[idx] = g_{best}[idx] \cdot (1 + 0.1 \cdot (\text{rand}() - 0.5)) \quad (25)$$

对于粒子 X，离散化全局时间轴（步长 0.01s），遍历每个时间点判定是否被任意弹遮挡，累加有效时间作为适应度：

$$\text{Fitness}(X) = T_{total} \quad (26)$$

对违反约束的粒子，赋予适应度=0，从而做到强制排除非可行解。

在约束条件下进行不断迭代优化，计算每个粒子的适应度，更新 p_{best} 和 g_{best} ，按改进策略更新速度与位置，修复违反约束的粒子，直至迭代次数 ≥ 300 或全局最优稳定（连续 30 次迭代变化 $\leq 1e-4$ ）。

5.4.3 问题三的结果

根据改进后的 POS 算法^[6]求解，我们可以得到 PSO 算法的适应度曲线呈现稳定收敛趋势，如下图 7 所示：

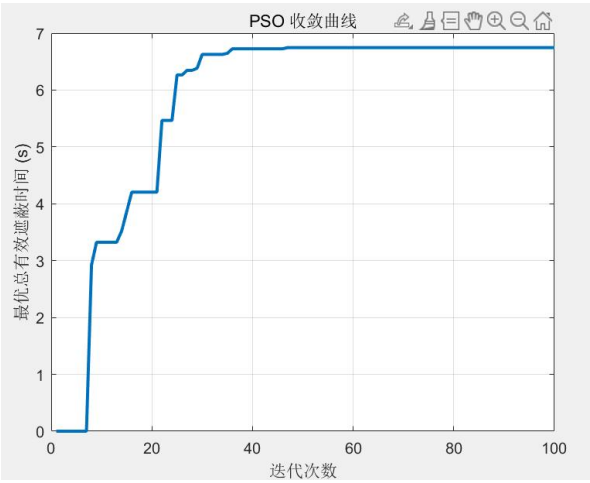


图 7：基于改进 PSO 算法优化过程的 T_{shield} 迭代收敛曲线

经改进粒子群算法优化求解，得到无人机 FY1 投放三枚烟幕弹协同干扰导弹 M1 的最优策略如表 1 所示：

表 1：三枚烟幕干扰弹的投放策略

无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹编号	烟幕干扰弹投放点的 x 坐标 (m)	烟幕干扰弹投放点的 y 坐标 (m)	烟幕干扰弹投放点的 z 坐标 (m)	烟幕干扰弹起爆点的 x 坐标 (m)	烟幕干扰弹起爆点的 y 坐标 (m)	烟幕干扰弹起爆点的 z 坐标 (m)	有效干扰时长 (s)
180.49	127.95	1	17800	0	1800	17674	-1.0641	1795.3	3.083
180.49	127.95	2	16514	-10.892	1800	15634	-18.347	1568.1	3.664
180.49	127.95	3	13190	-39.047	1800	11966	-49.409	1351.9	0

计算得出三枚烟幕干扰弹在最优策略下的相关时间如表 2 所示：

表 2：三枚烟幕干扰弹投放时间表

烟幕干扰弹	投放时刻 (s)	延迟 (s)	有效时长 (s)	遮蔽区间 (s)
1	0.000	0.982	3.083	[7.769,10.852]
2	10.052	6.880	3.664	[16.992,20.655]
3	36.033	9.563	0.000	-
总	-	-	6.7467	-

单无人机三弹对单弹作战情况下的烟幕时序图^[6]如图 8 所示，最优策略使用

三维状态仿真模拟如图 9 所示：

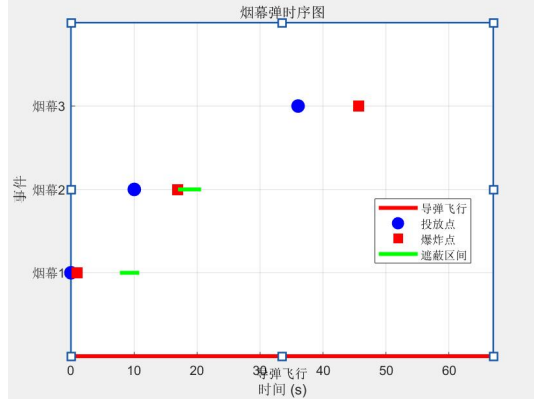


图 8: 单机三弹对单弹情况下的烟幕时序图

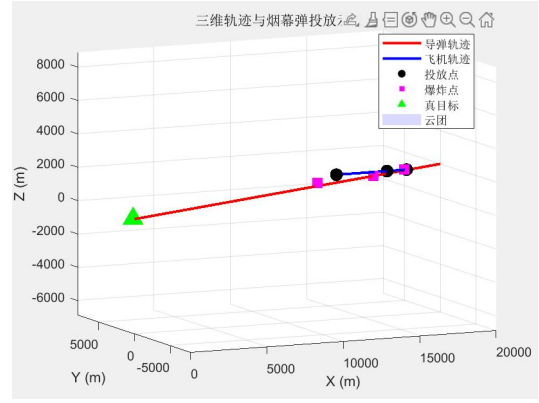


图 9: 单机三弹对单弹情况下的三维仿真图

5.5 问题四的模型求解与建立

5.5.1 问题四的求解思路

问题四是一个多无人机协同干扰的资源配置优化问题。基于问题三的单机多弹模型，我们将此问题转换为三架无人机在独立决策前提下实现遮蔽效果最优协同的优化问题。首先，我们以三枚烟幕弹总有效遮蔽时间并集最大化为目标函数，以各无人机速度范围、弹道物理规律为约束条件，建立以三架无人机的飞行方向、速度、投放时间及起爆延时等 12 个参数为决策变量的高维优化模型。最后，通过改进粒子群优化算法求解，得到使三机协同干扰效果最优的资源配置方案。

5.5.2 问题四的模型建立过程

5.5.2.1 数学模型的建立

问题四在问题二的基础上，由单机对单弹投放多枚烟幕干扰弹变成了多机对单弹投放单弹，求解时需同时确定三架无人机（FY1、FY2、FY3）各自的最佳飞行参数及弹道参数，通过协同配合最大化对导弹 M1 的联合遮蔽效果。由于三机独立且只携带一枚烟幕弹，针对该问题，我们忽略对烟幕弹的编号，定义无人机 FYK 的速度为 v_{FYk} 、方向角为 θ_k 、三枚弹的投放时间 $t_{rel,k}$ 、起爆延时为 Δt_k ，起爆时刻为 $t_{exp,k}$ 。

已知 D_{FYk} 为无人机 k 的初始位置，FY1(17800, 0, 1800)、FY2(12000, 1400, 1400)、FY3(6000, -3000, 700)。

则无人机 k 发射的烟幕干扰弹之间相互独立，各不干扰，其运动仍均分为两个阶段：平抛阶段和云团下沉阶段。在投放时刻 $t_{rel,k}$ 到起爆时刻 $t_{exp,k}$ 之间为平抛运动，投放点可表示为：

$$P_{drop,k} = F_k(t_{rel,k}) = \begin{cases} D_{FYk,x} + v_{FY1} \cdot t_{rel,k} \cdot \cos\theta_k \\ D_{FYk,y} + v_{FY1} \cdot t_{rel,k} \cdot \sin\theta_k \\ D_{FYk,z} \end{cases} \quad (27)$$

由于惯性，烟幕干扰弹在投放时刻之后，水平方向上仍保持无人机的速度 v_{FYk} 做匀速运动，在竖直方向上做自由落体运动，则起爆点坐标 $P_{exp,k}$ 可以表示为：

$$P_{exp,k} = \begin{cases} P_{drop,k,x} + v_{FY1} \cdot \Delta t_k \cdot \cos\theta_k \\ P_{drop,k,y} + v_{FY1} \cdot \Delta t_k \cdot \sin\theta_k \\ D_{FYk,z} - \frac{1}{2} g \Delta t_k^2 \end{cases} \quad (28)$$

起爆之后，烟幕干扰弹瞬时形成球状烟幕云团，由于巨大的阻力，云团失去水平方向的速度，竖直方向以 3m/s 匀速下沉，则云团中心的轨迹可以表示为：

$$C_k(t) = \begin{cases} P_{exp,k,x} \\ P_{exp,k,y} \\ P_{exp,k,z} - v_{sink} \cdot (t - t_{exp,k}) \end{cases} \quad (29)$$

导弹 M1 的轨迹，判断有效遮蔽与前序模型一致。

同问题三一样，在多弹情况下总遮蔽时间为单弹遮蔽时间的并集，可设全局时间设全局时间轴范围为 $[t_{global,min}, t_{global,max}]$ ， $(t_{global,min} = \min(t_{exp,1}, t_{exp,2}, t_{exp,3}), t_{global,max} = \max(t_{exp,1} + 20, t_{exp,2} + 20, t_{exp,3} + 20), t_{arrive})$ ，取步长为 0.01s 离散化全局时间轴，生成 $t_{list,global}$ 。遍历每一个 t ，如果存在 k 使得 k 弹遮蔽有效，则累加 0.01s 继续遍历，则时间并集的总时长 T_{total} 可表示为：

$$T_{total} = \sum_{t \in t_{list,global}} I\left(\bigcup_{k=1}^3 (k \text{ 弹遮蔽有效})\right) \times 0.01 \quad (30)$$

其中， $I(\cdot)$ 为指示函数，遮蔽时间有效时取 1，否则取 0。

优化目标为最大化三枚烟幕干扰弹的总有效遮蔽时长，目标函数可表示为：

$$\max T_{total} = (\theta_1, \theta_2, \theta_3, v_1, v_2, v_3, t_{rel,1}, t_{rel,2}, t_{rel,3}, \Delta t_1, \Delta t_2, \Delta t_3) \quad (31)$$

5.5.2.2 数学模型的求解

在问题二的基础上，依然使用粒子群优化算法求解目标函数，当无人机数量增加到三个之后，决策变量变为 12 个，每个粒子对应一组可行解，此时的向量结构 X 及初始化范围为：

$$X = [\theta_1, \theta_2, \theta_3, v_1, v_2, v_3, t_{rel,1}, t_{rel,2}, t_{rel,3}, \Delta t_1, \Delta t_2, \Delta t_3] \quad (32)$$

$(\theta \in [0, 2\pi], v_{FY1} \in [70, 140], t_{rel,k} \geq 0, \Delta t_1 \in [10^{-6}, 15], k = 1, 2, 3)$

算法依然采用采用动态惯性权重策略，设置 PSO 参数值如下：

表 3：PSO 算法相关参数设置

参数名称	参数值/参数范围	设置依据
粒子数量 N	120	高维空间需要更多粒子覆盖
最大迭代次数	1000	维度增加需更多迭代确保收敛
惯性权重 w	0.8~0.2 动态衰减+0.05 随机扰动	避免过早收敛
学习因子 c_1, c_2	2, 2	
最大速度约束 ($\theta_k, v_k, t_{rel,k}, \Delta t_k$)	$[\frac{\pi}{4}, 10, 5, 1] \times 3$	防止变量更新过快，确保收敛稳定性

设置时间并集总时长 T_{total} 为适应度函数。按照 PSO 优化算法^[7]迭代流程，首先随机生成 120 个粒子并初始化其位置和速度，设置 p_{best} 和 g_{best} ，然后计算每个粒子的适应度值并更新 p_{best} 和 g_{best} ；然后按照动态惯性权重策略更新每个粒子的速度和位置，此处为了确保收敛稳定性，我们对决策变量 $\theta_k, v_k, t_{rel,k}, \Delta t_k$ 进行最大速度约束，航向角变化限幅 $\frac{\pi}{4}$ ，速度变化限幅为每次迭代 10 m/s，时间参数变化限幅为每次迭代 5 s，延时参数变化限幅为 1 s。经过 250 代迭代优化；接着修复违反约束的粒子；若连续 25 次迭代 g_{best} 适应度变化 $\leq 1e-4$ ，对 3 枚弹的起爆延迟（低敏感度变量）添加 $\pm 10\%$ 变异，帮助跳出局部最优；当迭代次数 ≥ 1000 ，输出 g_{best} 。

5.5.3 问题四的结果

根据上述算法，我们可以得到最有投放策略如表 4 所示：

表 4：三架无人机的干扰弹最优投放策略

无人机编号	无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹投放点的 x 坐标 (m)	烟幕干扰弹投放点的 y 坐标 (m)	烟幕干扰弹投放点的 z 坐标 (m)	烟幕干扰弹起爆点的 x 坐标 (m)	烟幕干扰弹起爆点的 y 坐标 (m)	烟幕干扰弹起爆点的 z 坐标 (m)	有效干扰时长 (s)
FY1	8.91	70	17800	0	1800	17871.9	11.27	1794.7	4.76
FY2	285.5	132.32	12176.2	763.51	1400	12374.7	46.16	1244.9	4.86
FY3	93.88	114.99	5827.61	-456.4	700	5790.18	95.81	586.46	4.92
总									14.52

根据改进后的 POS 算法求解，得到的迭代收敛曲线如下图 10 所示：

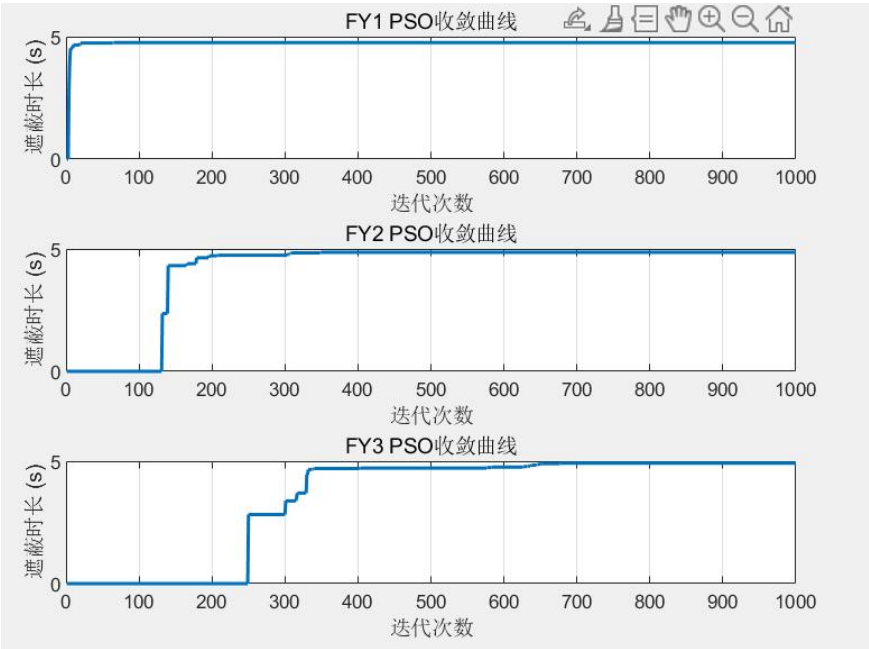


图 10：三枚烟幕干扰弹的 PSO 迭代收敛曲线

如图 11 和图 12 所示，我们利用 Matlab 绘制出在三架无人机各自发射一枚干扰弹情况下的导弹与无人机的三维轨迹图和导弹与遮蔽段的三维轨迹：

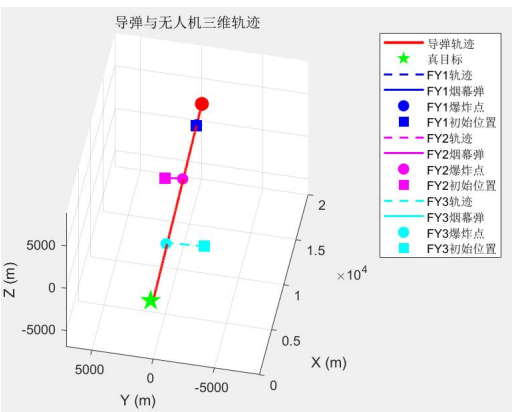


图 11：导弹与无人机的三维轨迹仿真图

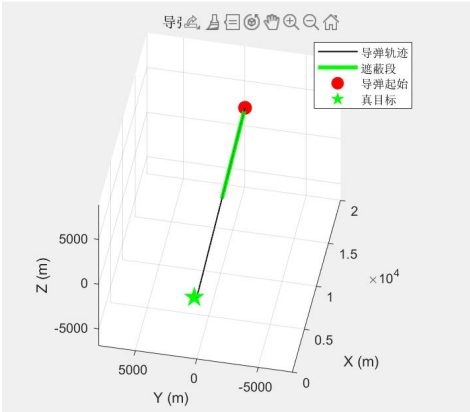


图 12：导弹与遮蔽段的三维轨迹仿真图

同时，我们还绘制了如图 13 所示的烟幕遮蔽状态时序图，从图中可以看出三枚烟幕干扰弹分别在不同时段发挥遮蔽作用：

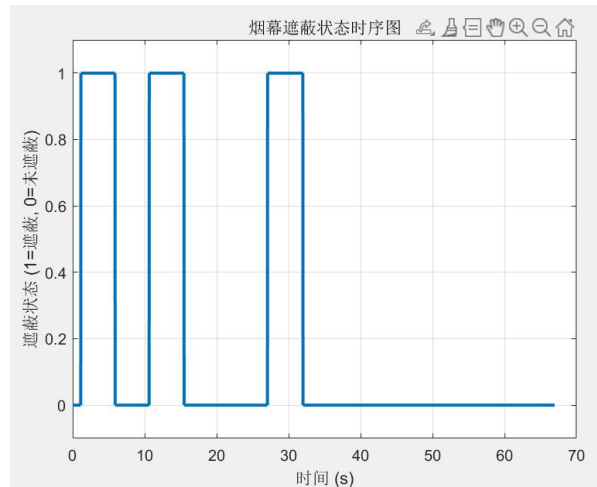


图 13：多机单弹对单弹情况下的烟幕遮蔽时序图

5.6 问题五的模型求解与建立

5.6.1 问题五的求解思路

问题五在问题四的基础上增加了导弹个数，从确保单一导弹被遮蔽变为追求三枚导弹被遮蔽的时间交集，这需要我们增加约束条件，要求 M1、M2、M3 均被至少 1 枚烟幕弹遮蔽时才是有效遮蔽，同时无人机飞机增多至 5 架且每架可投最多 3 枚弹，决策变量显著增多。要解决此问题，我们首先要通过分层编码将高维决策变量分解为无人机飞行方向与速度、烟幕弹投放与起爆参数、以及干扰分配关系三类子变量；接着采用离散-连续混合优化策略，对干扰分配关系使用 0-1 规划确保每枚导弹至少被一枚弹覆盖，对连续变量采用 PSO 粒子群算法优化飞行与弹道参数；最后机弹道模拟、遮蔽时间计算与全局协同优化逐步逼近最大化三枚导弹同时被遮蔽的总有效时长。

5.6.2 问题五的模型建立过程

导弹运动轨迹方程、无人机运动轨迹方程和烟幕弹运动轨迹方程可继承问题二、问题三、问题四中建立的模型，此时由于时多机多弹对多弹问题，在判断遮蔽判定条件时需要重新定义“遮蔽时间”：即某一时刻 t ，若 M1、M2、M3 均被至少 1 枚烟幕弹（所有烟幕弹的并集）遮蔽，则 t 计入总时长；反之不计入，且总时长基于烟幕弹有效遮蔽时刻的 3 枚导弹同时遮蔽交集计算。

此时决策变量数量骤增，我们将其分为连续变量和离散变量，连续变量包括：5 架无人机的飞行速度、方向角、投放时间和起爆时间，8 维离散变量包括素有时机投放的烟幕弹的干扰目标分配。面对这种超维、非线性、混合整数规划问题，本文采用一种“分治-协同”混合优化策略进行求解，仅修正适应度函数和候选筛选逻辑。将复杂的联合优化问题分解为三个层次：

在单机多弹优化阶段，依然采用改进的粒子群算法（PSO），将适应度函数修改为“该无人机的弹对 3 枚导弹的‘同时遮蔽贡献’”——即若该无人机的弹能覆盖某枚导弹，计 1 分，覆盖 3 枚计 3 分，对无人机的飞行参数和烟幕弹的时序参数进行并行优化。

$$X = \begin{bmatrix} \theta_1, \dots, \theta_5; v_1, \dots, v_5; t_{rel,1,1}, \dots, t_{rel,5,3}; \\ \Delta t_{1,1}, \dots, \Delta t_{5,3}; S_1, S_2, S_3; n_1, \dots, n_5 \end{bmatrix} \quad (33)$$

单弹对单弹的遮蔽判定和问题四相同，设导弹到达地面的时刻为 $t_{arrive,i}$ ，单枚烟幕干扰弹 (k,j) 对导弹 i 遮蔽时刻有效时需满足 $t \in \{t_{exp,k,j}, k, j, \min(t_{exp,k,j}, k, j + 20, t_{arrive,i})\}$ 且满足“导弹 i -真目标 Q ”视线线段到 $C_{kj}(t)$ 的最短距离不超过 $10m$ 。

设单枚导弹 j 的遮蔽时刻集合为 T_i ，则 T_i 可表示为：

$$T_i = \bigcup_{(k,j) \in S_i} \{t | t \in \{t_{exp,k,j}, k, j, \min(t_{exp,k,j}, k, j + 20, t_{arrive,i})\}\} \quad (34)$$

总有效遮蔽时长 T_{total} 为3枚导弹屏蔽时刻的交集，按照时间离散步长 $0.02s$ 叠加，

$$T_{total} = \sum_{t \in T_{global}} I(t \in T_1 \cap t \in T_2 \cap t \in T_3) \times 0.02 \quad (35)$$

其中， $I(\cdot)$ 为指示函数， t 同时在 T_1, T_2, T_3 中时取1，否则取0。

则目标函数为：

$$\max T_{total} \left(\begin{bmatrix} \theta_1, \dots, \theta_5; v_1, \dots, v_5; t_{rel,1,1}, \dots, t_{rel,5,3}; \\ \Delta t_{1,1}, \dots, \Delta t_{5,3}; S_1, S_2, S_3; n_1, \dots, n_5 \end{bmatrix} \right) \quad (36)$$

约束条件：

$$\left\{ \begin{array}{l} \theta_k \in [0, 2\pi] \\ v_{FYk} \in [70, 140] \\ t_{rel,kj} \geq 0, t_{rel,kj} \geq t_{rel,kj-1} + 1 \\ \Delta t_{kj} \geq 0.01 \\ t_{exp,kj} + 20 \leq t_{arrive,i} \\ P_{exp,kj,z} = P_{drop,kj} - \frac{1}{2} g \Delta t_{kj}^2 \\ n_k \in \{0, 1, 2, 3\} \text{ 且 } \sum n_k \geq 3 \\ S_i \neq \emptyset \end{array} \right. \quad (37)$$

在多机多导弹协同阶段，采用0-1整数规划和贪心组合筛选结合的方法。设0-1变量 $x_{k,i,j}$ ($1 \leq k \leq 5, 1 \leq i, j \leq 3$)，则当 $x_{k,i,j} = 1$ 表示烟幕干扰弹 (k,j) 对导弹 i 干扰；当 $x_{k,i,j} = 0$ 表示烟幕干扰弹 (k,j) 对导弹 i 不干扰。则要求每一枚干扰弹至多干扰三枚导弹： $x_{k,1,j} + x_{k,2,j} + x_{k,3,j} \leq 1$ ；每枚导弹至少被一枚干扰弹干扰： $\sum_{k=1}^5 \sum_{j=1}^3 x_{k,i,j} \geq 1$ ；仅投弹才参与干扰：若 $n_k < j$ ，则 $x_{k,i,j} = 0$ 。

则此时目标函数可表示为：

$$\max T_{total}(x_{k,i,j}, \text{连续参数}) \quad (38)$$

在第三阶段采用并行局部爬山算法，按照无人机分组进行优化，聚焦“延长三枚导弹的交集时刻”。

5.6.3 问题五的结果

算出最后结果为25s，遮蔽状态图如图14所示：

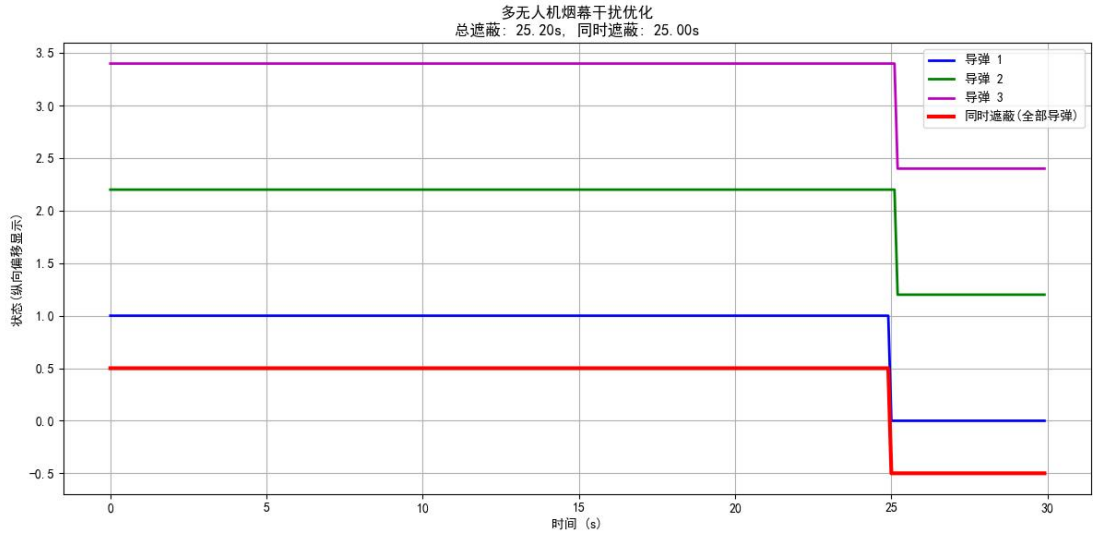


图 14: 遮蔽状态图

六、模型总结

6.1 模型优点

(1) 本文采用三维直角坐标系对导弹、无人机、烟幕弹及云团的运动轨迹进行精确建模,通过建立包含空间位置、速度向量和弹道参数的综合运动学方程,准确描述了各实体在三维空间中的动态交互过程,确保了遮蔽效能评估的准确性和可靠性。

(2) 本文采用 PSO 粒子群优化算法求解复杂优化问题。相比于传统优化算法,PSO 能够同时评估种群中多个粒子的适应度值,不需要计算目标函数的梯度信息,仅需调整种群大小和迭代次数等少量参数即可获得满意解,使得模型能够在合理时间内处理多无人机协同干扰这类高维优化问题。

(3) 本文对于几个问题的模型建立具有可传递性,从问题一的固定参数计算,到问题二的单变量优化,再到问题三的多弹时序协同和问题四的多智能体协同,本文建立了一个层层递进、由简至繁的模型体系,通过增加决策变量和使模型由特殊到一般化,使模型更加有层次。

(4) 本文针对高维优化时易"早熟收敛"的难点,引入了动态惯性权重与随机扰动机制对 PSO 算法进行改进。该策略有效提升了算法全局搜索能力,这从优化结果的显著提升和收敛曲线的稳定性上得到了验证。

6.2 模型缺点

(1) 本文模型是在理想状况下讨论的,未充分考虑环境、无人机飞行状况、弹药情况等因素的影响,可能导致实际遮蔽效果与理论计算存在偏差。

(2) 当决策变量逐渐增多时,解空间呈指数级增长,尽管采用 PSO 算法并且加以改进,但在有限迭代次数内难以保证找到全局最优,需要增加迭代次数并修改模拟的粒子数量。

6.3 模型推广

（1）本文建立的三维运动学模型、烟幕扩散模型及优化算法可推广应用于现代战场的其他电子对抗领域，为信息化作战提供理论支撑。

（2）本文采用的改进粒子群优化算法可推广至其他军事运筹学问题的求解中，如无人机集群路径规划、多目标火力分配等问题，为解决复杂军事决策问题提供有效工具。

七、参考文献

- [1] 刘静梅, 肖昌达. 烟幕遮蔽干扰反导作战使用[J]. 舰船电子对抗, 2007, (03):20-22. DOI:10.16426/j.cnki.jcdzdk.2007.03.005.
- [2] 罗瑞耀, 王得霖, 罗威, 等. 烟幕弹应对察打一体无人机的投放策略研究[J]. 光电技术应用, 2022, 37(06):90-98.
- [3] 顾文慧, 高红杰. 烟幕投放控制分析与设计[J]. 光电技术应用, 2014, 29(02):87-89.
- [4] 崔津, 柴洁, 耿荣妹, 等. 基于改进粒子群算法的无人机集群路径规划[J/OL]. 自动化技术与应用, 1-7[2025-09-07].
- [5] 才干. 直升机机载无源干扰的战术算法研究[J]. 光电对抗与无源干扰, 2000, (02):27-31.
- [6] 豆包, 4.0 版本, 字节跳动公司, 2025-09-05.
- [7] Gpt, 5.0free, open_ai, 2025-09-06

八、附录

附录 1

介绍：使用论文求解工具

论文写作软件 :Microsoft Word、WPS

流程图绘制软件::X-mind、WPS

编程工具:PyCharm、MATLAB

使用 Python 库:NumPy、Pandas、Matplotlib、Time

附录 2

介绍：支撑材料的文件列表

AI 工具使用详情.doc

question1.mat

question2.py

question3.mat

question4.mat

question5.py

result1.xlsx

result2.xlsx

result3.xlsx

附录 3

介绍：该代码是 matlab 语言编写的，作用解决问题一，并可视化内容

```
clear;clc;
```

```
g=9.8;
```

```
Vm=300;
```

```
Vu=120;
```

```
t_rel=1.5;
```

```
t_del=3.6;
```

```
t_exp = t_rel + t_del;
```

```
% 真目标
```

```
P = [0, 200, 0];
```

```
% M1 初始位置
```

```
Dij = [20000, 0, 2000];
```

```
% 无人机 FY1 初始位置
```

```
DFY1 = [17800, 0, 1800];
```

```

% 假目标(原点)
O = [0, 0, 0];

%% 导弹方向
um = (O - Dij) / norm(O - Dij);

%% 无人机速度向量(朝向原点, x 轴负方向)
uu = (O - DFY1); uu(3) = 0; uu = uu / norm(uu); % 等高度, 只取水平向量
uu = uu * Vu;

%% 投放点
d_rel = DFY1 + uu * t_rel;

%% 起爆点
dt = t_del;
d_exp = d_rel + uu * dt + 0.5 * [0, 0, -g] * dt^2;

%% 云团中心轨迹
%% c(t) for t >= t_explosion
c = @(t) d_exp + [0, 0, -3] * (t - t_exp);

%% 导弹轨迹
rm = @(t) Dij + Vm * t * um;
%% 点到线段距离函数
distPoint2Segment = @(P,A,B)...
    norm(P - (A + max(0,min(1, dot(P-A,B-A)/norm(B-A)^2))*(B-A)));

%% 扫描时间区间
t_ran = t_exp:0.01:t_exp+20;
mask = false(size(t_ran));
for k = 1:length(t_ran)
    t = t_ran(k);
    A = rm(t); B = P; L = c(t);
    d = distPoint2Segment(L,A,B);
    if d <= 10
        mask(k) = true;
    end
end

%% 找出连续区间
d_mask = diff([false mask false]);
start_idx = find(d_mask==1);
end_idx = find(d_mask==-1)-1;

intervals = [t_ran(start_idx)', t_ran(end_idx)'];
durations = intervals(:,2)-intervals(:,1);
total_duration = sum(durations);

```

```

%% 输出结果
disp('有效遮蔽时间区间(s):');
disp(intervals);
fprintf('总有效遮蔽时长 = %.3f s\n', total_duration);
%% 绘图演示（清晰版，含云团轨迹）
figure; hold on; grid on; axis equal
xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');
title('三维轨迹演示：导弹、无人机与云团遮蔽');
view(45,30);

%真目标
plot3(P(1),P(2),P(3),'rp','MarkerSize',14,'MarkerFaceColor','r');
text(P(1),P(2),P(3)+200,'真目标');

%导弹轨迹
t_m = linspace(0,t_exp+20,500);
M_traj = Dij + (Vm .* t_m(:)) * um;
plot3(M_traj(:,1),M_traj(:,2),M_traj(:,3),'r--','LineWidth',2);
plot3(Dij(1),Dij(2),Dij(3),'rs','MarkerFaceColor','r');
text(Dij(1),Dij(2),Dij(3)+200,'导弹初始');

% 无人机轨迹
t_fy = linspace(0,t_exp,200);
FY_traj = DFY1 + t_fy(:)*uu + 0.5*(t_fy(:).^2)*[0 0 -g];
plot3(FY_traj(:,1),FY_traj(:,2),FY_traj(:,3),'b','LineWidth',2);
plot3(d_exp(1),d_exp(2),d_exp(3),'bo','MarkerFaceColor','c');
text(d_exp(1),d_exp(2),d_exp(3)+200,'爆炸点');

% 云团轨迹（中心点随时间下落）
t_c = linspace(t_exp,t_exp+20,400);
C_traj = zeros(length(t_c),3);
for i = 1:length(t_c)
    C_traj(i,:) = d_exp + [0 0 -3]*(t_c(i)-t_exp);
end
plot3(C_traj(:,1),C_traj(:,2),C_traj(:,3),'g','LineWidth',2);
plot3(C_traj(1,1),C_traj(1,2),C_traj(1,3),'go','MarkerFaceColor','g');
text(C_traj(1,1),C_traj(1,2),C_traj(1,3)+200,'云团起点');

% 遮蔽区间高亮
for ii = 1:size(intervals,1)
    tt = linspace(intervals(ii,1),intervals(ii,2),100);
    Cm = zeros(length(tt),3);
    for j = 1:length(tt)
        Cm(j,:) = d_exp + [0 0 -3]*(tt(j)-t_exp);
    end
    plot3(Cm(:,1),Cm(:,2),Cm(:,3),'m','LineWidth',4);
end

```

```

legend('真目标','导弹轨迹','导弹初始','无人机轨迹','爆炸点','云团轨迹','云团
起点','有效遮蔽段');
legend('真目标','导弹轨迹','导弹初始','无人机轨迹','爆炸点','云团轨迹','云团
起点','有效遮蔽段');

%设置 X 轴以 10^4 为单位
ax = gca;
ax.XAxis.Exponent = 4;

```

附录 4

介绍：该代码是 python 语言编写的，作用是解决问题二，并可视化

```

import numpy as np
# 常量
D_ij = np.array([20000, 0, 2000])
D_FY1 = np.array([17800, 0, 1800])
v_M = 300
g = 9.8
v_z = 3
r = 10
C = np.array([0, 200, 0])
N = 100
MaxIter = 300
w_max, w_min = 0.8, 0.2
c1, c2 = 2, 2
dt = 0.02
# 初始化粒子
particles = np.zeros((N, 4))
particles[:, 0] = np.random.uniform(-np.pi, np.pi, N) # 飞行方向
particles[:, 1] = np.random.uniform(70, 140, N) # 飞行速度
particles[:, 2] = np.random.uniform(0, 67, N) # 投放时刻
particles[:, 3] = np.random.uniform(0.5, 10, N) # 起爆延时
velocities = np.random.uniform(-0.1, 0.1, (N, 4))
pbest = particles.copy()
pbest_fit = np.zeros(N)
gbest = particles[0].copy()
gbest_fit = -np.inf
u_M = -D_ij / np.linalg.norm(D_ij)
# 计算点到线段的距离
def dist_point(P, A, B):
    AP = P - A
    AB = B - A
    t = np.dot(AP, AB) / np.dot(AB, AB)
    t = max(0, min(1, t))
    closest = A + t * AB
    return np.linalg.norm(P - closest)
# 计算适应度

```



```

def fitness(particle):
    theta, v_fy1, t_rel, delta_t = particle
    t_det = t_rel + delta_t
    if t_det > 67: # 投放时刻过晚, 返回 0
        return 0, None, None # 适应度为 0, 且投放和起爆点为 None
    if t_rel == 0: # 不允许投放时刻为 0 秒
        return 0, None, None

    drop_pos = D_FY1 + v_fy1 * np.array([np.cos(theta), np.sin(theta), 0]) * t_rel
    det_x = drop_pos[0] + v_fy1 * delta_t * np.cos(theta)
    det_y = drop_pos[1] + v_fy1 * delta_t * np.sin(theta)
    det_z = drop_pos[2] - 0.5 * g * delta_t ** 2
    valid_time = 0
    t_list = np.arange(t_det, min(t_det + 20, 67), dt)
    for t in t_list:
        missile_pos = D_ij + v_M * u_M * t
        cloud_z = det_z - v_z * (t - t_det)
        cloud_pos = np.array([det_x, det_y, cloud_z])
        if dist_point(cloud_pos, missile_pos, C) <= r:
            valid_time += dt
    return valid_time, (drop_pos[0], drop_pos[1], drop_pos[2]), (det_x, det_y,
det_z)

# PSO 主循环
for iter in range(MaxIter):
    w = w_max - (w_max - w_min) * iter / MaxIter
    for i in range(N):
        fit, _, _ = fitness(particles[i]) # 解包适应度、投放点和起爆点
        if fit > pbest_fit[i]:
            pbest_fit[i] = fit
            pbest[i] = particles[i].copy()
        if fit > gbest_fit:
            gbest_fit = fit
            gbest = particles[i].copy()

    for i in range(N):
        r1, r2 = np.random.rand(2)
        velocities[i] = (w * velocities[i] +
                        c1 * r1 * (pbest[i] - particles[i]) +
                        c2 * r2 * (gbest - particles[i]))
        particles[i] += velocities[i]
        particles[i, 0] = particles[i, 0] % (2 * np.pi)
        particles[i, 1] = np.clip(particles[i, 1], 70, 140)
        particles[i, 2] = np.clip(particles[i, 2], 0, 67)
        particles[i, 3] = np.maximum(particles[i, 3], 0.5)
        if particles[i, 2] + particles[i, 3] > 67:
            particles[i, 2] = 67 - particles[i, 3]

# 输出最优结果
best_fitness, best_drop_pos, best_detonate_pos = fitness(gbest)

```

```

print("最优方向角(°):", np.degrees(gbest[0]))
print("最优无人机速度(m/s):", gbest[1])
print("最优投放时刻(s):", gbest[2])
print("最优起爆延迟(s):", gbest[3])
print("最大有效遮蔽时长(s):", best_fitness)
print("投放点坐标 (X, Y, Z):", best_drop_pos)
print("起爆点坐标 (X, Y, Z):", best_detonate_pos)

```

附录 5

介绍：该代码是 matlab 语言编写的，作用是解决问题三，并可视化

```

clc; clear; close all;

%% 初始参数
D_ij = [20000, 0, 2000]; % 导弹初始位置
v_M = 300; % 导弹速度
D_FY1 = [17800, 0, 1800]; % 飞机 1 初始位置
g = 9.8; % 重力加速度
v_z = 3; % 云团下沉速度
r = 10; % 云团半径
C = [0, 200, 0]; % 真目标位置

u_M = -D_ij / norm(D_ij); % 导弹运动方向向量

%% 粒子群算法参数
num_bombs = 3; % 烟幕弹数量
N = 80; % 粒子数量
MaxIter = 100; % 最大迭代数
w_max = 0.8; w_min = 0.2; % 惯性权重范围
c1 = 2; c2 = 1.5; % 学习因子

% 约束
t_rel_max = 60; % 最大投放时间
v_min = 70; v_max = 140; % 飞机速度范围
dt_min = 0.1; dt_max = 10; % 延迟时间范围

% 评估采样数量（在窗口 [t_det, t_det+20] 内）
Nt_eval = 1000;

%% 初始化粒子
% X = [theta, v_fy, t1, t2, t3, dt1, dt2, dt3]
particles = zeros(N, 8);
particles(:, 1) = rand(N, 1) * 2 * pi; % theta 角 [0, 2pi)
particles(:, 2) = v_min + (v_max - v_min) * rand(N, 1); % 飞机速度

% 随机生成三个投放时刻，保证相邻间隔 >= 1s
for i = 1:N

```

```

        t1 = rand * (t_rel_max - 2);
        lb2 = t1 + 1; ub2 = t_rel_max - 1;
        t2 = lb2 + rand * (ub2 - lb2);
        lb3 = t2 + 1; ub3 = t_rel_max;
        t3 = lb3 + rand * (ub3 - lb3);
        particles(i,3:5) = [t1, t2, t3];
    end

% 随机延迟时间
particles(:,6:8) = dt_min + (dt_max-dt_min).*rand(N,3);

% 初始化速度
velocities = (rand(N,8)-0.5)*0.2;

% 个体最优和全局最优
pbest = particles;
pbest_fit = -inf(N,1);
gbest = particles(1,:);
gbest_fit = -inf;
best_history = zeros(MaxIter,1);

%% PSO 主循环
for iter = 1:MaxIter
    % 惯性权重线性递减
    w = w_max - (w_max - w_min) * (iter-1)/(MaxIter-1);

    for i = 1:N
        X = particles(i,:);
        fit = fitness8(X, D_ij, v_M, u_M, D_FY1, g, v_z, r, C, Nt_eval);

        % 更新个体最优
        if fit > pbest_fit(i)
            pbest_fit(i) = fit;
            pbest(i,:) = X;
        end

        % 更新全局最优
        if fit > gbest_fit
            gbest_fit = fit;
            gbest = X;
        end
    end
    best_history(iter) = gbest_fit;

    % 更新速度和位置
    for i = 1:N
        r1 = rand; r2 = rand;
        velocities(i,:) = w*velocities(i,:) ...
            + c1*r1*(pbest(i,:) - particles(i,:)) ...

```

```

        + c2*r2*(gbest - particles(i,:));

particles(i,:) = particles(i,:) + velocities(i,:);

% 约束处理
particles(i,1) = mod(particles(i,1), 2*pi); % theta 周期约束
particles(i,2) = min(max(particles(i,2), v_min), v_max); % 速度约束

% 投放时间约束（在 [0,t_release_max] 且间隔>=1s）
tvals = sort(max(min(particles(i,3:5), t_rel_max), 0));
t1 = tvals(1);
t2 = max(tvals(2), t1 + 1);
t3 = max(tvals(3), t2 + 1);

% 如果超出范围就整体往左收缩
if t3 > t_rel_max
    t3 = t_rel_max;
    t2 = max(t3 - 1, t1 + 1);
    t1 = max(t2 - 1, 0);
end
particles(i,3:5) = [t1, t2, t3];

% 延迟时间约束
particles(i,6:8) = min(max(particles(i,6:8), dt_min), dt_max);
end
end

%% 输出最优解
theta = gbest(1);
v_fy = gbest(2);
tvals = gbest(3:5);
dtvals = gbest(6:8);

[total_time, bomb_intervals, drop_pts, det_pts] = collection( ...
    gbest, D_ij, v_M, u_M, D_FY1, g, v_z, r, C, Nt_eval);

% 结果表格
data = zeros(num_bombs,11);
intervals_str = cell(num_bombs,1);
durations = zeros(num_bombs,1);

for k = 1:num_bombs
    drop = drop_pts(k,:);
    det = det_pts(k,:);
    intervals_k = bomb_intervals{k};
    if isempty(intervals_k)
        intervals_str{k} = '';
        durations(k) = 0;
    else

```

```

        s = ""; dur = 0;
        for j=1:size(intervals_k,1)
            t0 = intervals_k(j,1);
            t1 = intervals_k(j,2);
            s = [s, sprintf('%.3f,%.3f' , t0, t1)];
            dur = dur + (t1 - t0);
        end
        intervals_str{k} = strtrim(s);
        durations(k) = dur;
    end
    data(k,:) = [k, rad2deg(theta), v_fy, tvals(k), dtvals(k), drop, det];
end

T = array2table(data, 'VariableNames', { ...
'index','theta_deg','uav_speed','t_rel','dt','drop_x','drop_y','drop_z','det_x','det_y','det_z'});
T.total_effective_time = repmat(total_time, num_bombs, 1);
T.individual_intervals = intervals_str;
T.individual_durations = durations;

disp('==== 三枚烟幕弹投放方案总结 ====');
disp(T);

fprintf('\n 三枚烟幕弹总有效遮蔽时间(并集): %.4f s\n\n', total_time);
for k=1:num_bombs
    fprintf('烟幕  %d: 投放时刻=%.3f s, 延迟=%.3f s, 单独有效时长=%.3f s,
遮蔽区间: %s\n', ...
        k, tvals(k), dtvals(k), durations(k), T.individual_intervals{k});
end

% 收敛曲线
figure;
plot(1:MaxIter, best_history, 'LineWidth',2);
xlabel('迭代次数'); ylabel('最优总有效遮蔽时间 (s)');
title('PSO 收敛曲线');
grid on;

%% 新增图形：时序图
figure;
hold on;
grid on;
title('烟幕弹时序图');
xlabel('时间 (s)');
ylabel('事件');

% 计算导弹飞行总时间
t_missile_total = norm(D_ij - C) / v_M;

```

```

% 绘制导弹飞行时间线
plot([0, t_missile_total], [0, 0], 'r-', 'LineWidth', 3);
text(t_missile_total/2, -0.2, '导弹飞行', 'HorizontalAlignment', 'center');

% 绘制每个烟幕弹的投放和爆炸事件
for k = 1:num_bombs
    t_rel = tvals(k);
    t_det = t_rel + dtvals(k);

    % 投放时刻
    plot(t_rel, k, 'bo', 'MarkerSize', 10, 'MarkerFaceColor', 'b');
    text(t_rel, k+0.2, sprintf('投放%d: %.1fs', k, t_rel), 'FontSize', 10);

    % 爆炸时刻
    plot(t_det, k, 'rs', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
    text(t_det, k+0.2, sprintf('爆炸%d: %.1fs', k, t_det), 'FontSize', 10);

    % 遮蔽区间
    intervals_k = bomb_intervals{k};
    if ~isempty(intervals_k)
        for j = 1:size(intervals_k,1)
            t_start = intervals_k(j,1);
            t_end = intervals_k(j,2);
            plot([t_start, t_end], [k, k], 'g-', 'LineWidth', 3);
        end
    end
end

% 设置坐标轴范围
xlim([0, max(t_missile_total, max(tvals)+max(dtvals)+20)]);
ylim([0, num_bombs+1]);
set(gca, 'YTick', 1:num_bombs, 'YTickLabel', {'烟幕 1', '烟幕 2', '烟幕 3'});
legend('导弹飞行', '投放点', '爆炸点', '遮蔽区间', 'Location', 'best');

%% 新增图形：三维轨迹图
figure;
hold on;
grid on;
axis equal;
view(3);
title('三维轨迹与烟幕弹投放示意图');
xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');

% 计算导弹轨迹
t_missile = linspace(0, t_missile_total, 100);
missile_pos = D_ij + v_M * u_M .* t_missile;
plot3(missile_pos(:,1), missile_pos(:,2), missile_pos(:,3), 'r-', 'LineWidth', 2);

```

```

% 计算飞机轨迹
vvec = v_fy * [cos(theta), sin(theta), 0];
t_plane = linspace(0, max(tvals), 100);
plane_pos = D_FY1 + vvec .* t_plane';
plot3(plane_pos(:,1), plane_pos(:,2), plane_pos(:,3), 'b-', 'LineWidth', 2);

% 绘制投放点
scatter3(drop_pts(:,1), drop_pts(:,2), drop_pts(:,3), 100, 'k', 'filled', 'Marker', 'o');

% 绘制爆炸点
scatter3(det_pts(:,1), det_pts(:,2), det_pts(:,3), 100, 'm', 'filled', 'Marker', 's');

% 绘制真目标
scatter3(C(1), C(2), C(3), 200, 'g', 'filled', 'Marker', '^');

% 绘制云团（在爆炸点位置）
[xs, ys, zs] = sphere(20);
for k = 1:3
    cloud_x = det_pts(k,1) + r * xs;
    cloud_y = det_pts(k,2) + r * ys;
    cloud_z = det_pts(k,3) + r * zs;
    surf(cloud_x, cloud_y, cloud_z, 'FaceAlpha', 0.3, 'EdgeColor', 'none',
'FaceColor', [0.5, 0.5, 1]);
end

% 添加图例
legend('导弹轨迹', '飞机轨迹', '投放点', '爆炸点', '真目标', '云团', 'Location',
'best');

%%子函数

function fit = fitness8(X, D_ij, v_M, u_M, D_FY1, g, v_z, r, C, Nt_eval)
    [total_time, ~, ~, ~] = collection(X, D_ij, v_M, u_M, D_FY1, g, v_z, r, C,
Nt_eval);
    fit = total_time;
end

function [total_time, bomb_intervals, drop_pts, det_pts] = collection( ...
    X, D_ij, v_M, u_M, D_FY1, g, v_z, r, C, Nt_eval)
    theta = X(1); v_fy = X(2);
    tvals = X(3:5); dtvals = X(6:8);

    vvec = v_fy * [cos(theta), sin(theta), 0];

    bomb_intervals = cell(3,1);
    drop_pts = zeros(3,3);
    det_pts = zeros(3,3);
    all_intervals = [];

```

```

for k = 1:3
    t_rel = tvals(k);
    dt_k = dtvals(k);
    t_det = t_rel + dt_k;

    drop_pos = D_FY1 + vvec * t_rel;
    det_z = drop_pos(3) - 0.5 * g * dt_k^2;
    det_x = drop_pos(1) + vvec(1) * dt_k;
    det_y = drop_pos(2) + vvec(2) * dt_k;
    det_pos = [det_x, det_y, det_z];

    drop_pts(k,:) = drop_pos;
    det_pts(k,:) = det_pos;

    t_start = t_det;
    t_end = t_det + 20;
    if t_end <= t_start
        bomb_intervals{k} = [];
        continue;
    end
    t_list = linspace(t_start, t_end, Nt_eval);
    valid_flags = false(size(t_list));

    for ii = 1:length(t_list)
        tt = t_list(ii);
        missile_pos = D_ij + v_M * u_M * tt;
        cloud_z = det_pos(3) - v_z * (tt - t_det);
        cloud_pos = [det_pos(1), det_pos(2), cloud_z];

        line_vec = missile_pos - C;
        denom = dot(line_vec, line_vec);
        if denom == 0, continue; end
        s = dot(cloud_pos - missile_pos, line_vec) / denom;
        s = max(0, min(1, s));
        closest = missile_pos + s * line_vec;
        if norm(cloud_pos - closest) <= r
            valid_flags(ii) = true;
        end
    end

    intervals_k = flags_to_intervals(t_list, valid_flags);
    bomb_intervals{k} = intervals_k;
    if ~isempty(intervals_k)
        all_intervals = [all_intervals; intervals_k];
    end
end
total_time = union_time(all_intervals);
end

function inter = flags_to_intervals(t_list, flags)

```



```

inter = [];
if ~any(flags), return; end
idx = find(flags);
d = [true, diff(idx)~=1];
starts = idx(d);
ends_mask = [diff(idx)~=1, true];
ends = idx(ends_mask);
for i = 1:length(starts)
    sidx = starts(i); eidx = ends(i);
    inter = [inter; t_list(sidx), t_list(eidx)];
end
end

function total = union_time(intervals)
    if isempty(intervals)
        total = 0; return;
    end
    intervals = sortrows(intervals,1);
    merged = intervals(1,:);
    for i = 2:size(intervals,1)
        if intervals(i,1) <= merged(end,2) + eps*10
            merged(end,2) = max(merged(end,2), intervals(i,2));
        else
            merged = [merged; intervals(i,:)];
        end
    end
    total = sum(merged(:,2) - merged(:,1));
end

```

附录 5

介绍：该代码是 matlab 语言编写的，作用是解决问题四，并可视化

```

clear; close all; clc;
rng('shuffle');

%% 常量定义
D_M1 = [20000, 0, 2000];           % 导弹初始位置
v_M = 300;                         % 导弹速度
t_arrive = norm(D_M1) / v_M;        % 大约 67 s
C = [0, 200, 0];                   % 真目标代表点

% 无人机初始位置
D_FY{1} = [17800, 0, 1800];
D_FY{2} = [12000, 1400, 1400];
D_FY{3} = [6000, -3000, 700];

% 烟幕参数
g = 9.8;
v_sink = 3;
r = 10;
dt = 0.02;                         % 时间分辨率

```

```

% PSO 与并行参数
N_PARTICLE = 120;    % 粒子数
MAX_ITER = 1000;     % 迭代次数
K_CANDIDATE = 10;
M_TOP_COMBO = 5;
LOCAL_ITER = 50;

% 并行设置
poolobj = gcp('nocreate');
if isempty(poolobj)
    parpool; % 若没开启则默认开启
end

%% 生成单机候选方案（PSO）
fprintf('生成 FY1 候选方案...\n');
[fy1_candidates, gbest_history1] = uav_pso(1, N_PARTICLE, MAX_ITER,
K_CANDIDATE);
fprintf('生成 FY2 候选方案...\n');
[fy2_candidates, gbest_history2] = uav_pso(2, N_PARTICLE, MAX_ITER,
K_CANDIDATE);
fprintf('生成 FY3 候选方案...\n');
[fy3_candidates, gbest_history3] = uav_pso(3, N_PARTICLE, MAX_ITER,
K_CANDIDATE);

% 绘制 PSO 迭代曲线
figure('Name', 'PSO 迭代收敛曲线');
subplot(3,1,1);
plot(1:length(gbest_history1), gbest_history1, 'LineWidth', 1.5);
title('FY1 PSO 收敛曲线');
xlabel('迭代次数');
ylabel('遮蔽时长 (s)');
grid on;

subplot(3,1,2);
plot(1:length(gbest_history2), gbest_history2, 'LineWidth', 1.5);
title('FY2 PSO 收敛曲线');
xlabel('迭代次数');
ylabel('遮蔽时长 (s)');
grid on;

subplot(3,1,3);
plot(1:length(gbest_history3), gbest_history3, 'LineWidth', 1.5);
title('FY3 PSO 收敛曲线');
xlabel('迭代次数');
ylabel('遮蔽时长 (s)');
grid on;

```

```

%% 筛选三机协同潜力组合
fprintf('筛选三机协同潜力组合...\n');
top_combos = [];
idx = 0;
for i1 = 1:length(fy1_candidates)
    for i2 = 1:length(fy2_candidates)
        for i3 = 1:length(fy3_candidates)
            idx = idx + 1;
            p1 = fy1_candidates{i1}.params;
            p2 = fy2_candidates{i2}.params;
            p3 = fy3_candidates{i3}.params;
            total_time = joint_time(p1, p2, p3, D_M1, v_M, C, D_FY, g, v_sink,
r, dt, t_arrive);
            top_combos(idx).p1 = p1; %#ok<SAGROW>
            top_combos(idx).p2 = p2;
            top_combos(idx).p3 = p3;
            top_combos(idx).total = total_time;
        end
    end
end

% 排序并选择 top M
[~, order] = sort([top_combos.total], 'descend');
order = order(1:min(M_TOP_COMBO, numel(order)));
potential_combos = top_combos(order);

%% 并行局部精修潜力组合
fprintf('并行局部精修潜力组合...\n');
refined_combos = cell(1, numel(potential_combos));
parfor ci = 1:numel(potential_combos)
    combo = potential_combos(ci);
    % 按顺序对三枚弹做局部精修
    new1 = local_refine(combo.p1, 1, {combo.p1, combo.p2, combo.p3},
LOCAL_ITER, D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive);
    new2 = local_refine(combo.p2, 2, {new1, combo.p2, combo.p3},
LOCAL_ITER, D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive);
    new3 = local_refine(combo.p3, 3, {new1, new2, combo.p3}, LOCAL_ITER,
D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive);
    total = joint_time(new1, new2, new3, D_M1, v_M, C, D_FY, g, v_sink, r, dt,
t_arrive);
    refined_combos{ci}.p1 = new1;
    refined_combos{ci}.p2 = new2;
    refined_combos{ci}.p3 = new3;
    refined_combos{ci}.total = total;
end

% 收集并排序
totals = cellfun(@(x) x.total, refined_combos);

```

```

[~, ord2] = sort(totals, 'descend');
best = refined_combos{ord2(1)};
best_time = best.total;
p1 = best.p1; p2 = best.p2; p3 = best.p3;

%% 输出最优方案
fprintf('\n===== 最优三机协同遮蔽方案 =====\n');
fprintf('Total_time: %.2f s\n', best_time);
for k = 1:3
    if k==1, p = p1; elseif k==2, p = p2; else p = p3; end
    fprintf('\n--- FY%d 参数 ---\n', k);
    fprintf('V: %.2f m/s\n', p(1));
    fprintf('theta: %.2f °\n', rad2deg(p(2)));
    fprintf('drop_time: %.2f s\n', p(3));
    fprintf('delay_time: %.2f s\n', p(4));
    st = single_time(p, k, D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive);
    fprintf('time: %.2f s\n', st);
end
%% 输出表格数据
fprintf('\n===== 表格数据输出 =====\n');
fprintf('无人机编号\t运动方向(°)\t速度(m/s)\t投放点 x(m)\t投放点 y(m)\t投放点 z(m)\t起爆点 x(m)\t起爆点 y(m)\t起爆点 z(m)\t有效干扰时长(s)\n');

for k = 1:3
    if k==1
        p = p1;
        D0 = D_FY{1};
    elseif k==2
        p = p2;
        D0 = D_FY{2};
    else
        p = p3;
        D0 = D_FY{3};
    end

    v = p(1);
    theta = p(2);
    t_rel = p(3);
    delta_t = p(4);

    % 计算投放点坐标
    drop_x = D0(1) + v * cos(theta) * t_rel;
    drop_y = D0(2) + v * sin(theta) * t_rel;
    drop_z = D0(3);

    % 计算起爆点坐标
    det_x = drop_x + v * cos(theta) * delta_t;
    det_y = drop_y + v * sin(theta) * delta_t;
    det_z = max(drop_z - 0.5 * g * delta_t^2, 0);

```

```

% 计算单机有效干扰时长
st = single_time(p, k, D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive);

% 输出表格行

fprintf('FY%d\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\n', ...
        k, rad2deg(theta), v, drop_x, drop_y, drop_z, det_x, det_y, det_z, st);
end

%% 可视化
% 1. 绘制时序图
figure('Name', '烟幕遮蔽时序图');
[~, mask] = joint_time(p1, p2, p3, D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive,
true);
t_span = 0:dt:(t_arrive - dt);
plot(t_span, mask, 'LineWidth', 2);
title('烟幕遮蔽状态时序图');
xlabel('时间 (s)');
ylabel('遮蔽状态 (1=遮蔽, 0=未遮蔽)');
ylim([-0.1 1.1]);
grid on;

% 2. 绘制三维轨迹图
figure('Name', '导弹与无人机三维轨迹');
hold on;
grid on;
axis equal;
view(3);

% 绘制导弹轨迹
missile_traj = zeros(floor(t_arrive/dt)+1, 3);
for i = 1:length(t_span)
    t = t_span(i);
    missile_traj(i,:) = D_M1 + v_M * (-D_M1 / norm(D_M1)) * t;
end
plot3(missile_traj(:,1), missile_traj(:,2), missile_traj(:,3), 'r-', 'LineWidth', 2);

% 绘制真目标
plot3(C(1), C(2), C(3), 'gp', 'MarkerSize', 15, 'MarkerFaceColor', 'g');

% 绘制无人机轨迹和烟幕弹
colors = ['b', 'm', 'c'];
for k = 1:3
    if k==1, p = p1; D0 = D_FY{1}; color = colors(1);
    elseif k==2, p = p2; D0 = D_FY{2}; color = colors(2);
    else p = p3; D0 = D_FY{3}; color = colors(3); end

```

```

v = p(1); theta = p(2); t_rel = p(3); delta_t = p(4);

% 无人机飞行轨迹 (到投放点)
drop_pos = D0 + v * [cos(theta), sin(theta), 0] * t_rel;
plot3([D0(1), drop_pos(1)], [D0(2), drop_pos(2)], [D0(3), drop_pos(3)], ...
      [color '--'], 'LineWidth', 1.5);

% 烟幕弹飞行轨迹 (投放点到爆炸点)
det_x = drop_pos(1) + v * delta_t * cos(theta);
det_y = drop_pos(2) + v * delta_t * sin(theta);
det_z = max(drop_pos(3) - 0.5 * g * delta_t^2, 0);
plot3([drop_pos(1), det_x], [drop_pos(2), det_y], [drop_pos(3), det_z], ...
      [color '-'], 'LineWidth', 1.5);

% 标记爆炸点
plot3(det_x, det_y, det_z, [color 'o'], 'MarkerSize', 8, 'MarkerFaceColor',
color);

% 标记无人机初始位置
plot3(D0(1), D0(2), D0(3), [color 's'], 'MarkerSize', 10, 'MarkerFaceColor',
color);
end

% 标记导弹初始位置
plot3(D_M1(1), D_M1(2), D_M1(3), 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');

% 添加图例和标题
legend('导弹轨迹', '真目标', 'FY1 轨迹', 'FY1 烟幕弹', 'FY1 爆炸点', 'FY1 初始位置', ...
      'FY2 轨迹', 'FY2 烟幕弹', 'FY2 爆炸点', 'FY2 初始位置', ...
      'FY3 轨迹', 'FY3 烟幕弹', 'FY3 爆炸点', 'FY3 初始位置', ...
      'Location', 'bestoutside');
title('导弹与无人机三维轨迹');
xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');

% 3. 绘制遮蔽段导弹轨迹
figure('Name', '导弹轨迹与遮蔽段');
hold on;
grid on;
axis equal;
view(3);

% 绘制整个导弹轨迹
plot3(missile_traj(:,1), missile_traj(:,2), missile_traj(:,3), 'k-', 'LineWidth', 1);

% 绘制遮蔽段
shield_indices = find(mask);
if ~isempty(shield_indices)

```

```

        plot3(missile_traj(shield_indices,1), missile_traj(shield_indices,2),
missile_traj(shield_indices,3), ...
            'g-', 'LineWidth', 3);
    end

    % 标记关键点
    plot3(D_M1(1), D_M1(2), D_M1(3), 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
    plot3(C(1), C(2), C(3), 'gp', 'MarkerSize', 15, 'MarkerFaceColor', 'g');

    % 添加图例和标题
    legend('导弹轨迹', '遮蔽段', '导弹起始', '真目标', 'Location', 'best');
    title('导弹轨迹与遮蔽段');
    xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');%% ----- 输出最优方
案 -----
    fprintf('\n===== 最优三机协同遮蔽方案 =====\n');
    fprintf('Total_time: %.2f s\n', best_time);
    for k = 1:3
        if k==1, p = p1; elseif k==2, p = p2; else p = p3; end
        fprintf('\n--- FY%d 参数 ---\n', k);
        fprintf('V: %.2f m/s\n', p(1));
        fprintf('theta: %.2f °\n', rad2deg(p(2)));
        fprintf('drop_time: %.2f s\n', p(3));
        fprintf('delay_time: %.2f s\n', p(4));
        st = single_time(p, k, D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive);
        fprintf('time: %.2f s\n', st);
    end

    %% 本地函数

    function [candi, gbest_hist] = uav_pso(fy_id, n_particle, max_iter,
K_CANDIDATE)
        % PSO for single UAV to get candidate k best solutions
        % 参数顺序: [v, theta, t_rel, delta_t]
        particles = zeros(n_particle, 4);
        particles(:,1) = 70 + (140-70)*rand(n_particle,1); % v
        particles(:,2) = -pi + (2*pi)*rand(n_particle,1); % theta
        particles(:,3) = rand(n_particle,1) * evalin('base','t_arrive'); % t_rel (use base
workspace t_arrive)
        particles(:,4) = 0.01 + (10-0.01)*rand(n_particle,1); % delta_t
        velocities = -0.1 + (0.2).*rand(n_particle,4);
        pbest = particles;
        pbest_fit = -inf(n_particle,1);
        gbest = particles(1,:);
        gbest_fit = -inf;
        gbest_hist = zeros(1, max_iter); % 记录每代最优值

```

```

        for iter = 1:max_iter
            w = 0.8 - (0.8 - 0.2) * (iter-1) / max_iter;
            for i = 1:n_particle
                fit = single_time(particles(i,:), fy_id, evalin('base','D_M1'),
evalin('base','v_M'), evalin('base','C'), evalin('base','D_FY'), evalin('base','g'),
evalin('base','v_sink'), evalin('base','r'), evalin('base','dt'), evalin('base','t_arrive'));
                if fit > pbest_fit(i)
                    pbest_fit(i) = fit;
                    pbest(i,:) = particles(i,:);
                end
                if fit > gbest_fit
                    gbest_fit = fit;
                    gbest = particles(i,:);
                end
            end
            gbest_hist(iter) = gbest_fit; % 记录当前代最优值

            if mod(iter,50)==0
                fprintf('FY%d - 迭代 %d/%d, 当前最佳单弹时长: %.2f s\n', fy_id,
iter, max_iter, gbest_fit);
            end

            for i = 1:n_particle
                r1 = rand; r2 = rand;
                velocities(i,:) = w*velocities(i,:) + 2*r1*(pbest(i,:) - particles(i,:)) +
2*r2*(gbest - particles(i,:));
                particles(i,:) = particles(i,:) + velocities(i,:);
                % 边界约束
                particles(i,1) = min(max(particles(i,1), 70), 140);
                particles(i,2) = mod(particles(i,2), 2*pi);
                particles(i,3) = min(max(particles(i,3), 0), evalin('base','t_arrive'));
                particles(i,4) = min(max(particles(i,4), 0.01), 10);
            end
        end

        % 选取 top K_CANDIDATE pbest
        [~, idxs] = sort(pbest_fit, 'descend');
        k = min(K_CANDIDATE, numel(idxs));
        candi = cell(1,k);
        for i = 1:k
            candi{i}.params = pbest(idxs(i,:));
            candi{i}.fit = pbest_fit(idxs(i));
        end
    end

    function d = dist_point(P, A, B)
        % 计算点到线段的最短距离
        AP = P - A;
        AB = B - A;

```



```

        denom = dot(AB,AB) + 1e-8;
        t = dot(AP,AB) / denom;
        t = max(0, min(1, t));
        closest = A + t .* AB;
        d = norm(P - closest);
    end

    function valid_time = single_time(params, fy_id, D_M1, v_M, C, D_FY, g, v_sink,
r, dt, t_arrive)
        % 单枚弹遮蔽时长计算
        v = params(1); theta = params(2); t_rel = params(3); delta_t = params(4);
        t_det = t_rel + delta_t;
        if t_det > t_arrive
            valid_time = 0;
            return;
        end
        drop_pos = D_FY{fy_id} + v * [cos(theta), sin(theta), 0] * t_rel;
        det_x = drop_pos(1) + v * delta_t * cos(theta);
        det_y = drop_pos(2) + v * delta_t * sin(theta);
        det_z = max(drop_pos(3) - 0.5 * g * delta_t^2, 0);
        t_start = t_det;
        t_end = min(t_det + 20, t_arrive);
        t_list = t_start:dt:(t_end - 1e-12);
        valid_time = 0;
        for t = t_list
            missile_pos = D_M1 + v_M * (-D_M1 / norm(D_M1)) * t;
            cloud_z = det_z - v_sink * (t - t_det);
            cloud_pos = [det_x, det_y, cloud_z];
            if dist_point(cloud_pos, missile_pos, C) <= r
                valid_time = valid_time + dt;
            end
        end
    end
end

function [total_time, mask] = joint_time(params1, params2, params3, D_M1, v_M,
C, D_FY, g, v_sink, r, dt, t_arrive, return_mask)
    % 三枚弹协同的真实总遮蔽时长（逐点合并）
    if nargin < 13, return_mask = false; end
    t_span = 0:dt:(t_arrive - dt);
    shield_mask = false(size(t_span));
    params_all = {params1, params2, params3};

    for fy_id = 1:3
        params = params_all{fy_id};
        v = params(1); theta = params(2); t_rel = params(3); delta_t = params(4);
        t_det = t_rel + delta_t;
        if t_det > t_arrive
            continue;
        end
        drop_pos = D_FY{fy_id} + v * [cos(theta), sin(theta), 0] * t_rel;

```

```

det_x = drop_pos(1) + v * delta_t * cos(theta);
det_y = drop_pos(2) + v * delta_t * sin(theta);
det_z = max(drop_pos(3) - 0.5 * g * delta_t^2, 0);

for idx = 1:numel(t_span)
    t = t_span(idx);
    if ~(t_det <= t && t <= min(t_det + 20, t_arrive))
        continue;
    end
    missile_pos = D_M1 + v_M * (-D_M1 / norm(D_M1)) * t;
    cloud_z = det_z - v_sink * (t - t_det);
    cloud_pos = [det_x, det_y, cloud_z];
    if dist_point(cloud_pos, missile_pos, C) <= r
        shield_mask(idx) = true;
    end
end
end

total_time = sum(shield_mask) * dt;
if return_mask
    mask = shield_mask;
else
    mask = [];
end
end

function best_params = local_refine(params, fy_id, target_params, LOCAL_ITER,
D_M1, v_M, C, D_FY, g, v_sink, r, dt, t_arrive)
% 简单的局部随机搜索精修
best_params = params;
% 计算当前适应度 (joint)
tp = target_params;
if fy_id == 1, tp{1} = best_params; end
if fy_id == 2, tp{2} = best_params; end
if fy_id == 3, tp{3} = best_params; end
best_fit = joint_time(tp{1}, tp{2}, tp{3}, D_M1, v_M, C, D_FY, g, v_sink, r,
dt, t_arrive);

for iter = 1:LOCAL_ITER
    improved = false;
    for i = 1:4
        step = -0.5 + rand*1.0; % uniform(-0.5,0.5)
        new_params = best_params;
        new_params(i) = new_params(i) + step;
        % 边界
        new_params(1) = min(max(new_params(1), 70), 140);
        new_params(2) = mod(new_params(2), 2*pi);
        new_params(3) = min(max(new_params(3), 0), t_arrive);
        new_params(4) = min(max(new_params(4), 0.01), 10);
        tp2 = target_params;

```

```

        if fy_id == 1, tp2{1} = new_params; end
        if fy_id == 2, tp2{2} = new_params; end
        if fy_id == 3, tp2{3} = new_params; end
        new_fit = joint_time(tp2{1}, tp2{2}, tp2{3}, D_M1, v_M, C, D_FY,
g, v_sink, r, dt, t_arrive);
        if new_fit > best_fit
            best_fit = new_fit;
            best_params = new_params;
            improved = true;
            break;
        end
    end
    if ~improved
        break;
    end
end
end

```

附录 6

介绍：用 matlab 编写，解决问题 5 并可视化

```

import numpy as np
import matplotlib.pyplot as plt
import itertools
import random
import matplotlib
import pandas as pd

# 设置中文字体
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans', 'Arial']
matplotlib.rcParams['axes.unicode_minus'] = False

# 核心参数配置
PARTICLE_COUNT = 100
MAX_STEPS = 150
CANDIDATE_NUM = 5
TOP_COMBINATIONS = 5
TIME_STEP = 0.1
MIN_ARRIVE_TIME = 30
SINK_VELOCITY = 0.5
EFFECT_RADIUS = 5.0
ARRIVE_TIMES = {1: 25, 2: 28, 3: 30}
MAX_SMOKES = 3
LAMBDA_FACTOR = 0.7

# 获取烟幕弹投放位置
def get_release_pos(params, drone_id):
    v, angle, release_t, delta_t = params
    x = drone_id * 10 + v * np.cos(angle) * release_t
    y = drone_id * 5 + v * np.sin(angle) * release_t

```

```

        z = 100 - SINK_VELOCITY * release_t
        return (x, y, z)

# 获取烟幕弹起爆位置
def get_detonate_pos(params, drone_id):
    v, angle, release_t, delta_t = params
    drop_x, drop_y, drop_z = get_release_pos(params, drone_id)
    det_x = drop_x + v * np.cos(angle) * delta_t
    det_y = drop_y + v * np.sin(angle) * delta_t
    det_z = drop_z - 0.5 * 9.8 * (delta_t ** 2)
    return (det_x, det_y, det_z)

# 导弹位置
def missile_pos(missile_id, t):
    x = 50 * missile_id - t * 2
    y = 30 * missile_id - t
    z = 50 + t
    return (x, y, z)

# 点到线段的距离计算
def point_segment_dist(point, start, end):
    point_2d = np.array(point[:2])
    start_2d = np.array(start[:2])
    end_2d = np.array(end[:2])
    vector = end_2d - start_2d
    t = max(0, min(1, np.dot(point_2d - start_2d, vector) / (np.linalg.norm(vector)
** 2 + 1e-8)))
    closest_point = start_2d + t * vector
    return np.linalg.norm(point_2d - closest_point)

# 计算单个烟幕的遮蔽效果
def single_smoke_shield(params, drone_id, missile_id):
    v, angle, release_t, delta_t = params
    detonate_t = release_t + delta_t
    if detonate_t > ARRIVE_TIMES[missile_id]:
        return 0.0, [] # 如果投放时刻过晚，直接返回 0

    release_pos = get_release_pos(params, drone_id)
    start_t = detonate_t
    end_t = min(detonate_t + 20, ARRIVE_TIMES[missile_id]) # 限制 20 秒有效时间窗

    shield_intervals = []

    for t in np.arange(start_t, end_t, TIME_STEP):
        missile_position = missile_pos(missile_id, t)
        cloud_height = release_pos[2] - SINK_VELOCITY * (t - detonate_t)
        cloud_center = (release_pos[0], release_pos[1], cloud_height)

        if point_segment_dist(cloud_center, missile_position, (0, 0, 0)) <=

```

```

EFFECT_RADIUS:
    shield_intervals.append(t)

    # 计算有效遮蔽时长，确保不会超过 20 秒
    valid_time = min(len(shield_intervals) * TIME_STEP, 20)

    return valid_time, shield_intervals

# 计算多个导弹的总遮蔽效果
def multi_missile_shield(smoke_params_list, return_details=False):
    time_range = np.arange(0, MIN_ARRIVE_TIME, TIME_STEP)
    coverage = {j: np.zeros_like(time_range, dtype=bool) for j in range(1, 4)}

    for drone_id, smokes in enumerate(smoke_params_list, start=1):
        for params in smokes:
            v, angle, release_t, delta_t = params
            detonate_t = release_t + delta_t
            rel_x, rel_y, rel_z = get_release_pos(params, drone_id)

            for mid in range(1, 4):
                if detonate_t > ARRIVE_TIMES[mid]:
                    continue

                start_idx = np.searchsorted(time_range, detonate_t)
                end_idx = np.searchsorted(time_range, min(detonate_t + 20,
ARRIVE_TIMES[mid])) # 限制 20 秒有效时窗

                for idx in range(start_idx, end_idx):
                    miss_pos = missile_pos(mid, time_range[idx])
                    cloud_z = rel_z - SINK_VELOCITY * (time_range[idx] -
detonate_t)

                    if point_segment_dist([rel_x, rel_y, cloud_z], miss_pos, (0,
0, 0)) <= EFFECT_RADIUS:
                        coverage[mid][idx] = True

                joint_coverage = coverage[1] & coverage[2] & coverage[3]
                joint_duration = np.sum(joint_coverage) * TIME_STEP
                total_coverage = coverage[1] | coverage[2] | coverage[3]
                total_duration = np.sum(total_coverage) * TIME_STEP
                score = LAMBDA_FACTOR * joint_duration + (1 - LAMBDA_FACTOR) *
total_duration

            if return_details:
                return score, joint_coverage, time_range, coverage, total_duration,
joint_duration
            else:
                return score

```

```

# 粒子群优化 (PSO)
def drone_pso(drone_id):
    particles = np.zeros((PARTICLE_COUNT, 2 + 2 * MAX_SMOKES))
    particles[:, 0] = np.random.uniform(70, 140, PARTICLE_COUNT)
    particles[:, 1] = np.random.uniform(-np.pi, np.pi, PARTICLE_COUNT)

    for i in range(MAX_SMOKES):
        particles[:, 2 + 2 * i] = np.random.uniform(0, MIN_ARRIVE_TIME,
PARTICLE_COUNT)
        particles[:, 3 + 2 * i] = np.random.uniform(0.01, 10,
PARTICLE_COUNT)

        velocities = np.random.uniform(-0.1, 0.1, particles.shape)
        personal_best = particles.copy()
        personal_scores = np.zeros(PARTICLE_COUNT)
        global_best = particles[0].copy()
        global_score = -np.inf

        for step in range(MAX_STEPS):
            inertia = 0.8 - (0.8 - 0.2) * step / MAX_STEPS

            for i in range(PARTICLE_COUNT):
                smokes = [particles[i, [0, 1, 2 + 2 * b, 3 + 2 * b]] for b in
range(MAX_SMOKES)]
                covers = []

                for mid in range(1, 4):
                    cov = sum(single_smoke_shield(p, drone_id, mid)[0] for p in
smokes)
                    covers.append(cov)

                fitness = sum(covers) + 0.5 * min(covers)

                if fitness > personal_scores[i]:
                    personal_scores[i] = fitness
                    personal_best[i] = particles[i].copy()

                if fitness > global_score:
                    global_score = fitness
                    global_best = particles[i].copy()

            for i in range(PARTICLE_COUNT):
                r1, r2 = np.random.rand(2)
                velocities[i] = (inertia * velocities[i] +
                                2 * r1 * (personal_best[i] - particles[i]) +
                                2 * r2 * (global_best - particles[i]))

                particles[i] += velocities[i]
                particles[i, 0] = particles[i, 0] % (2 * np.pi)
                particles[i, 1] = (particles[i, 1] + np.pi) % (2 * np.pi) - np.pi

```

```

        for b in range(MAX_SMOKEs):
            particles[i, 2 + 2 * b] = np.clip(particles[i, 2 + 2 * b], 0,
MIN_ARRIVE_TIME)
            particles[i, 3 + 2 * b] = np.clip(particles[i, 3 + 2 * b], 0.01, 10)

            if b > 0 and particles[i, 2 + 2 * b] < particles[i, 2 + 2 * (b - 1)] +
1:
                particles[i, 2 + 2 * b] = particles[i, 2 + 2 * (b - 1)] + 1

            top_indices = np.argsort(personal_scores)[-CANDIDATE_NUM:]
            return [(personal_best[i, [0, 1, 2 + 2 * b, 3 + 2 * b]] for b in
range(MAX_SMOKEs)], personal_scores[i]) for i in
                top_indices]

# 局部优化
def local_optimize(combo, iterations=20):
    best_combination = [[p.copy() for p in drone] for drone in combo]
    best_score = multi_missile_shield(best_combination)

    for _ in range(iterations):
        improved = False

        for u in range(len(best_combination)):
            for s in range(MAX_SMOKEs):
                for k in range(4):
                    step = (np.random.uniform(-0.5, 0.5) if k == 0 else
                        np.random.uniform(-0.05, 0.05) if k == 1 else
                        np.random.uniform(-0.2, 0.2) if k == 2 else
                        np.random.uniform(-0.1, 0.1))

                    new_combo = [[p.copy() for p in drone] for drone in
best_combination]

                    new_combo[u][s][k] += step
                    new_score = multi_missile_shield(new_combo)

                    if new_score > best_score:
                        best_score = new_score
                        best_combination = new_combo
                        improved = True
                        break

                    if improved: break

            if not improved:
                break

    return best_combination, best_score

```

```

# 主程序
def main():
    all_candidates = [drone_pso(did) for did in range(1, 6)]
    combo_list = []

    for combo in itertools.product(*all_candidates):
        current_combo = [c[0] for c in combo]
        combo_score = multi_missile_shield(current_combo)
        combo_list.append((current_combo, combo_score))

    combo_list.sort(key=lambda x: x[1], reverse=True)
    top_combos = [c[0] for c in combo_list[:TOP_COMBINATIONS]]

    refined_results = []
    for combo in top_combos:
        optimized_combo, score = local_optimize(combo)
        refined_results.append((optimized_combo, score))

    refined_results.sort(key=lambda x: x[1], reverse=True)
    best_combination, best_score = refined_results[0]

    score_details = multi_missile_shield(best_combination, return_details=True)
    score, joint_mask, time_span, masks, total_cover, joint_cover = score_details

    print(f'综合评分: {best_score:.2f}, 总遮蔽时间: {total_cover:.2f}s, 总体有效遮蔽时间: {joint_cover:.2f}s')

    data_rows = []
    for drone_id, smokes in enumerate(best_combination, start=1):
        drone_speed = smokes[0][0]
        drone_dir = np.degrees(smokes[0][1])
        print(f'\n 无人机 {drone_id} 速度: {drone_speed:.2f} m/s, 方向: {drone_dir:.2f}°')

        for smoke_idx, params in enumerate(smokes, start=1):
            drop_x, drop_y, drop_z = get_release_pos(params, drone_id)
            det_x, det_y, det_z = get_detonate_pos(params, drone_id)

            hit_missiles = []
            total_shield = 0.0
            for mid in range(1, 4):
                cov_time, _ = single_smoke_shield(params, drone_id, mid)
                if cov_time > 0:
                    hit_missiles.append(str(mid))
                total_shield += cov_time

            print(f'      烟 幕 弹      {smoke_idx}: 投 放 点
=({drop_x:.1f},{drop_y:.1f},{drop_z:.1f}), "
      f'起爆点=({det_x:.1f},{det_y:.1f},{det_z:.1f}), "

```



```

        f' 有效遮蔽时长={total_shield:.2f}s, 遮蔽导弹 :
        {'.'.join(hit_missiles) if hit_missiles else '无'}")

    data_rows.append({
        "无人机编号": drone_id,
        "无人机运动速度(m/s)": drone_speed,
        "无人机运动方向(°)": drone_dir,
        "烟幕弹编号": smoke_idx,
        "投放点 X(m)": drop_x,
        "投放点 Y(m)": drop_y,
        "投放点 Z(m)": drop_z,
        "起爆点 X(m)": det_x,
        "起爆点 Y(m)": det_y,
        "起爆点 Z(m)": det_z,
        "有效遮蔽时长(s)": total_shield,
        "遮蔽导弹编号": ",".join(hit_missiles)
    })

result_df = pd.DataFrame(data_rows)
result_df.to_csv("uav_smoke_middle_results.csv",          index=False,
encoding="utf-8-sig")
print("\n 中间过程数据已保存到 uav_smoke_middle_results.csv")

plt.figure(figsize=(12, 6))
offsets = {1: 0, 2: 1.2, 3: 2.4}
color_map = {1: 'b', 2: 'g', 3: 'm'}

for mid in range(1, 4):
    plt.plot(time_span, masks[mid].astype(int) + offsets[mid],
color_map[mid], lw=2, label=f'导弹 {mid}')

plt.plot(time_span, joint_mask.astype(int) - 0.5, 'r', lw=3, label='总体有效遮蔽
')

plt.xlabel("时间 (s)")
plt.ylabel("遮蔽状态(纵向偏移显示)")
plt.title(f'多无人机烟幕干扰优化\n 总遮蔽: {total_cover:.2f}s, 总体的有效
遮蔽: {joint_cover:.2f}s')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()

```